Abstract

This report studies dimensionality reduction and learned representations for five-class handwritten-digit classification. Principal component analysis (PCA) is evaluated at four retained dimensions. One-hidden-layer and three-hidden-layer autoencoders are trained to reconstruct the input images, and their bottleneck representations are evaluated through downstream fully connected neural-network (FCNN) classifiers. Denoising autoencoders trained with 20% and 40% corruption are assessed using both reconstruction mean squared error (MSE) and classification accuracy. Finally, input-to-bottleneck weight maps are visualized to compare the learned filters of a conventional autoencoder and the two denoising autoencoders. The report presents the experimental configurations, quantitative results, confusion matrices, reconstruction examples, and weight visualizations.

Keywords: PCA, autoencoder, denoising autoencoder, dimensionality reduction, representation learning, handwritten-digit classification, FCNN

Contents

I Introduction 2
II Dataset and Experimental Setup 2
III Task 1: PCA-Based Dimensionality Reduction 4
IV Task 2: Autoencoder Reconstruction 6
V Task 3: Classification Using One-Hidden-Layer Autoencoder Representations 19
VI Task 4: Classification Using Three-Hidden-Layer Autoencoder Representations 21
VII Task 5: Denoising Autoencoders 24
VIII Task 6: Input-to-Bottleneck Weight Visualization 28
IX Overall Comparison and Discussion 52
X Conclusion 53
References 53

List of Figures

1 Confusion matrix: confusion matrix pca for 32 components. 5
2 Confusion matrix: confusion matrix pca for 64 components. 5
3 Confusion matrix: confusion matrix pca for 128 components. 6
4 Confusion matrix: confusion matrix pca for 256 components. 6
5 Autoencoder reconstruction visualization: test 1 hidden with bottleneck of 32 neurons. 7
6 Autoencoder reconstruction visualization: test 1 hidden with bottleneck of 64 neurons. 8
7 Autoencoder reconstruction visualization: test 1 hidden with bottleneck of 128 neurons. 8
8 Autoencoder reconstruction visualization: test 1 hidden with bottleneck of 256 neurons. 9
9 Autoencoder reconstruction visualization: test 3 hidden with bottleneck of 32 neurons. 9
10 Autoencoder reconstruction visualization: test 3 hidden with bottleneck of 64 neurons. 10
11 Autoencoder reconstruction visualization: test 3 hidden with bottleneck of 128 neurons. 10
12 Autoencoder reconstruction visualization: test 3 hidden with bottleneck of 256 neurons. 11
13 Autoencoder reconstruction visualization: train 1 hidden with bottleneck of 32 neurons. 11
14 Autoencoder reconstruction visualization: train 1 hidden with bottleneck of 64 neurons. 12
15 Autoencoder reconstruction visualization: train 1 hidden with bottleneck of 128 neurons. 12
16 Autoencoder reconstruction visualization: train 1 hidden with bottleneck of 256 neurons. 13
17 Autoencoder reconstruction visualization: train 3 hidden with bottleneck of 32 neurons. 13
18 Autoencoder reconstruction visualization: train 3 hidden with bottleneck of 64 neurons. 14
19 Autoencoder reconstruction visualization: train 3 hidden with bottleneck of 128 neurons. 14
20 Autoencoder reconstruction visualization: train 3 hidden with bottleneck of 256 neurons. 15
21 Autoencoder reconstruction visualization: validation 1 hidden with bottleneck of 32 neurons. 15
22 Autoencoder reconstruction visualization: validation 1 hidden with bottleneck of 64 neurons. 16
23 Autoencoder reconstruction visualization: validation 1 hidden with bottleneck of 128 neurons. 16
24 Autoencoder reconstruction visualization: validation 1 hidden with bottleneck of 256 neurons. 17
25 Autoencoder reconstruction visualization: validation 3 hidden with bottleneck of 32 neurons. 17
26 Autoencoder reconstruction visualization: validation 3 hidden with bottleneck of 64 neurons. 18
27 Autoencoder reconstruction visualization: validation 3 hidden with bottleneck of 128 neurons. 18
28 Autoencoder reconstruction visualization: validation 3 hidden with bottleneck of 256 neurons. 19
29 Confusion matrix: confusion matrix for 32 neurons bottleneck. 20
30 Confusion matrix: confusion matrix for 64 neurons bottleneck. 20
31 Confusion matrix: confusion matrix for 128 neurons bottleneck. 21
32 Confusion matrix: confusion matrix for 256 neurons bottleneck. 21
33 Confusion matrix: confusion matrix for 32 neurons bottleneck. 22
34 Confusion matrix: confusion matrix for 64 neurons bottleneck. 23
35 Confusion matrix: confusion matrix for 128 neurons bottleneck. 23
36 Confusion matrix: confusion matrix for 256 neurons bottleneck. 24
37 Confusion matrix: confusion matrix for 20% noise. 25
38 Confusion matrix: confusion matrix for 40% noise. 25
39 Autoencoder reconstruction visualization: dae 20% noise test reconstructions. 26
40 Autoencoder reconstruction visualization: dae 20% noise train reconstructions. 26
41 Autoencoder reconstruction visualization: dae 20% noise validation reconstructions. 27
42 Autoencoder reconstruction visualization: dae 40% noise test reconstructions. 27
43 Autoencoder reconstruction visualization: dae 40% noise train reconstructions. 28
44 Autoencoder reconstruction visualization: dae 40% noise validation reconstructions. 28
45 Input-to-bottleneck weight visualization for neurons 1–16. 29
46 Input-to-bottleneck weight visualization for neurons 17–32. 30
47 Input-to-bottleneck weight visualization for neurons 33–48. 31
48 Input-to-bottleneck weight visualization for neurons 49–64. 32
49 Input-to-bottleneck weight visualization for neurons 65–80. 33
50 Input-to-bottleneck weight visualization for neurons 81–96. 34
51 Input-to-bottleneck weight visualization for neurons 97–112. 35
52 Input-to-bottleneck weight visualization for neurons 113–128. 36
53 Input-to-bottleneck weight visualization for neurons 1–16. 37
54 Input-to-bottleneck weight visualization for neurons 17–32. 38

55 Input-to-bottleneck weight visualization for neurons 33–48. 39
56 Input-to-bottleneck weight visualization for neurons 49–64. 40
57 Input-to-bottleneck weight visualization for neurons 65–80. 41
58 Input-to-bottleneck weight visualization for neurons 81–96. 42
59 Input-to-bottleneck weight visualization for neurons 97–112. 43
60 Input-to-bottleneck weight visualization for neurons 113–128. 44
61 Input-to-bottleneck weight visualization for neurons 1–16. 45
62 Input-to-bottleneck weight visualization for neurons 17–32. 46
63 Input-to-bottleneck weight visualization for neurons 33–48. 47
64 Input-to-bottleneck weight visualization for neurons 49–64. 48
65 Input-to-bottleneck weight visualization for neurons 65–80. 49
66 Input-to-bottleneck weight visualization for neurons 81–96. 50
67 Input-to-bottleneck weight visualization for neurons 97–112. 51
68 Input-to-bottleneck weight visualization for neurons 113–128. 52

List of Tables

I Validation accuracy for every FCNN architecture and PCA dimension. . . . 4
II Best FCNN result for each PCA dimension. . . . 4
III Reconstruction MSE on training, validation, and test sets after the selected 500-epoch runs. . . . 7
IV Validation accuracy for every FCNN architecture and one-hidden-layer AE representation. . . . 19
V Best classification result by one-hidden-layer AE representation. . . . 20
VI Validation accuracy for every FCNN architecture and three-hidden-layer AE representation. . . . 22
VII Best classification result by three-hidden-layer AE representation. . . . 22
VIII Task 5: Denoising reconstruction MSE. . . . 24
IX DAE classification results. . . . 25
X Overall best test classification results. . . . 53

I. Introduction

Dimensionality reduction represents an input using fewer features while attempting to preserve information relevant to a task. In this assignment, PCA provides a linear, data-driven projection, while autoencoders learn nonlinear representations by reconstructing their inputs through a bottleneck. The quality of a representation is considered from two perspectives: reconstruction fidelity and its usefulness for classification.

The experiments comprise six tasks: (1) PCA followed by FCNN classification, (2) reconstruction using one-hidden-layer and three-hidden-layer autoencoders, (3) classification using one-hidden-layer autoencoder representations, (4) classification using the encoder of the three-hidden-layer autoencoder, (5) denoising autoencoders at two noise levels, and (6) visualization of input-to-bottleneck weights. The report includes the supplied result figures and discusses the findings alongside the numerical results.

II. Dataset and Experimental Setup

A. Dataset

The experiments use five digit classes, labelled 1, 5, 6, 7, and 9. Each grayscale image is 28×28 pixels and is flattened into a 784-dimensional vector. The data split described in the Assignment 3 reference report contains 11,385 training examples, 3,795 validation examples, and 3,795 test examples. The same split description is used here; input preprocessing and split-specific transformations are described within the relevant task.

B. Software libraries

The experiments were implemented in Python. PyTorch (`torch`, `torch.nn`, and `torch.optim`) was used to define and train the FCNNs and autoencoders, compute losses, and perform optimization on the selected device. NumPy was used for numerical array operations, while scikit-learn (`sklearn.decomposition.PCA`) was used for PCA-based dimensionality reduction. Pandas was used to organize and export result tables, and Matplotlib was used to generate plots and confusion-matrix visualizations. Pillow (`PIL`) was used for image handling. Python's standard-library modules, including `os`, `random`, `glob`, `zipfile`, and `joblib` where needed, supported file management, reproducibility, and loading/saving experiment artifacts.

C. Evaluation and model selection

Classification performance is reported as accuracy, and confusion matrices are used to inspect class-wise predictions. Autoencoder reconstruction is evaluated using mean squared error (MSE). Validation results are used for model selection; test results are reported for final evaluation. Unless noted otherwise, percentages are reported as percentages rather than fractions.

D. Classification architectures

The experiments reuse four FCNN configurations from the preceding assignment: `3 hidden best`(represented as 3_hidden_best) with hidden widths [128, 64, 32], `4 hidden best`(represented as 4_hidden_best) with [128, 64, 32, 16], `5 hidden best`(represented as 5_hidden_best) with [256, 128, 64, 32, 16], and `4 hidden normal`(represented as 4_hidden_normal) with [256, 128, 64, 32]. The selected architecture can vary with the representation dimension. The Assignment 3 reference model achieved 98.1291% test accuracy.

E. Input normalization

The image intensities are represented on the interval $[0, 1]$. For an original pixel intensity p in the 8-bit range $[0, 255]$, normalization is

$$\mathbf{x} = \frac{\mathbf{p}}{255}. \tag{1}$$

The normalized 28×28 image is flattened to $\mathbf{x} \in \mathbb{R}^{784}$.

F. FCNN objective, activation, and optimization

For an FCNN layer, the affine transformation is $a^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$. The hidden layers use the hyperbolic tangent activation

$$\tanh(\mathbf{z}) = \frac{e^{\mathbf{z}} - e^{-\mathbf{z}}}{e^{\mathbf{z}} + e^{-\mathbf{z}}} \tag{2}$$

and the output layer produces five class logits $z_1, \ldots, z_5$. For target class y, the cross-entropy objective is

$$\mathcal{L}_{\mathrm{CE}} = -\log \frac{\exp(z_y)}{\sum_{k=1}^{5} \exp(z_k)}. \tag{3}$$

The FCNN weights use Xavier uniform initialization as defined in the weight-initialization subsection. The FCNNs are optimized with stochastic gradient descent with momentum, using learning rate $\eta = 0.001$, momentum coefficient $\mu = 0.9$, and batch size 1. The weight update is

$$\Delta \mathbf{w}(m) = -\eta \nabla \mathbf{w}(m) + \alpha \Delta \mathbf{w}(m-1), \qquad 0 < \alpha < 1. \tag{4}$$

The new weight is then obtained as

$$\mathbf{w}(m+1) = \mathbf{w}(m) + \Delta \mathbf{w}(m). \tag{5}$$

For all PCA and learned-representation classification experiments, the four FCNN architectures listed in Section 2.3 are compared. The maximum training budget is 100 epochs, and the best checkpoint is selected by validation accuracy. Early stopping is based on the absolute change in average training loss between successive epochs:

$$|\bar{\mathcal{L}}_t - \bar{\mathcal{L}}_{t-1}| < 10^{-4}. \tag{6}$$

The convergence check is disabled during the first two completed epochs. This stopping criterion is used for the FCNN classifiers, not for the autoencoder reconstruction runs.

G. Autoencoder objective, activations, and optimization

For input x and reconstruction $\hat{x}$, the autoencoder is trained to minimize mean squared reconstruction error,

$$\mathcal{L}_{\text{MSE}} = \frac{1}{ND} \sum_{n=1}^{N} \sum_{j=1}^{D} (x_{nj} - \hat{x}_{nj})^2, \qquad D = 784. \tag{7}$$

The autoencoder weights use Xavier uniform initialization as defined in the weight-initialization subsection. The one-hidden-layer autoencoders use a linear 128/64/32/256-dimensional bottleneck as specified for each run, logistic output activation, and Adam with learning rate 0.001. For the three-hidden-layer architecture $784 \to 400 \to d \to 400 \to 784$, the bottleneck is linear, the other hidden layers use logistic activation, and the output uses logistic activation. The autoencoder runs use 500 epochs and Adam with learning rate 0.001; the best checkpoint is selected using validation reconstruction loss. The denoising autoencoders use the same linear 128-dimensional bottleneck and logistic output, with Adam, learning rate 0.001, batch size 128, and a maximum of 500 epochs.

H. Weight initialization

FCNN and Autoencoder weights are initialized using Xavier uniform initialization. For a weight matrix with fan-in n_{in} and fan-out n_{out}, each weight is sampled as

$$W_{ij} \sim \mathcal{U}(-a, a), \qquad a = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}. \tag{8}$$

This is a zero-mean uniform distribution with variance $\text{Var}(W_{ij}) = 2/(n_{\text{in}} + n_{\text{out}})$.

I. PCA projection

Let $X_{\text{train}} \in \mathbb{R}^{N_{\text{train}} \times 784}$, $X_{\text{val}} \in \mathbb{R}^{N_{\text{val}} \times 784}$, and $X_{\text{test}} \in \mathbb{R}^{N_{\text{test}} \times 784}$ denote the normalized input matrices. The feature-wise mean is computed from the training set only:

$$\boldsymbol{\mu}_{\text{train}} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \mathbf{x}_i^{\text{train}}. \tag{9}$$

Each split is centered using this same training mean:

$$X_{\text{train}}^c = X_{\text{train}} - \mathbf{1}_{N_{\text{train}}} \boldsymbol{\mu}_{\text{train}}^\top, \tag{10}$$

$$X_{\text{val}}^c = X_{\text{val}} - \mathbf{1}_{N_{\text{val}}} \boldsymbol{\mu}_{\text{train}}^\top, \tag{11}$$

$$X_{\text{test}}^c = X_{\text{test}} - \mathbf{1}_{N_{\text{test}}} \boldsymbol{\mu}_{\text{train}}^\top. \tag{12}$$

Here, $\mathbf{1}_N$ is an N-dimensional column vector of ones; equivalently, the training mean is subtracted from every row of each data matrix. The covariance matrix is computed from the centered training data only:

$$C = \frac{(X_{\text{train}}^c)^\top X_{\text{train}}^c}{N_{\text{train}} - 1}. \tag{13}$$

Let $V_d \in \mathbb{R}^{784 \times d}$ contain the eigenvectors associated with the d largest eigenvalues of C. The PCA representations are then

$$Z_{\text{train}} = X_{\text{train}}^c V_d, \quad Z_{\text{val}} = X_{\text{val}}^c V_d, \quad Z_{\text{test}} = X_{\text{test}}^c V_d. \tag{14}$$

The training mean and projection directions are reused unchanged for validation and test data, avoiding data leakage from estimating split-specific means or projection directions.

III. Task 1: PCA-Based Dimensionality Reduction

A. Method

PCA is used to project the original 784-dimensional input vectors onto 32, 64, 128, or 256 principal components. The projection directions are derived from the training data. The training mean is used when centering the training, validation, and test data so that the validation and test transformations do not estimate their own means. FCNNs are then trained on each PCA representation using the architectures specified for the experiment.

B. Results

The assignment asks for validation accuracy across the tested FCNN architectures, so Table I reports all validation runs; Table II then summarizes the validation-selected model and its held-out test accuracy. The 64-dimensional and 128-dimensional representations tie for the highest test accuracy, at 98.2345%. The 32-dimensional representation is slightly lower, while the 256-dimensional representation does not improve on the best result despite retaining more components. Thus, increasing the number of components does not guarantee improved downstream classification.

TABLE I
Validation accuracy for every FCNN architecture and PCA dimension.

Representation dim.	FCNN architecture	Validation accuracy (%)
32	4_hidden_normal	98.6825
32	5_hidden_best	98.6298
32	3_hidden_best	98.4980
32	4_hidden_best	98.3663
64	5_hidden_best	98.5771
64	3_hidden_best	98.4980
64	4_hidden_normal	98.3926
64	4_hidden_best	98.2872
128	4_hidden_normal	98.4190
128	4_hidden_best	98.3926
128	3_hidden_best	98.3399
128	5_hidden_best	98.3136
256	5_hidden_best	98.4453
256	4_hidden_best	98.4190
256	4_hidden_normal	98.4190
256	3_hidden_best	98.2082

TABLE II
Best FCNN result for each PCA dimension.

PCA dim.	Architecture	Val. acc. (%)	Test acc. (%)
32	4-hidden normal	98.6825	98.0501
64	5-hidden best	98.5771	98.2345
128	4-hidden normal	98.4190	98.2345
256	5-hidden best	98.4453	98.1291

Compared with the Assignment 3 baseline (98.1291% test accuracy), PCA improves test accuracy by 0.1049 percentage points at both 64 and 128 dimensions. The 64-D model uses `5_hidden_best`, while the 128-D model uses `4_hidden_normal`; both achieve 98.2345%. Thus, PCA is competitive with the original 784-D FCNN while using fewer input features.

The confusion matrices for all four dimensions follow. They show the detailed class-level distribution of correct predictions and errors and complement the aggregate accuracy values.

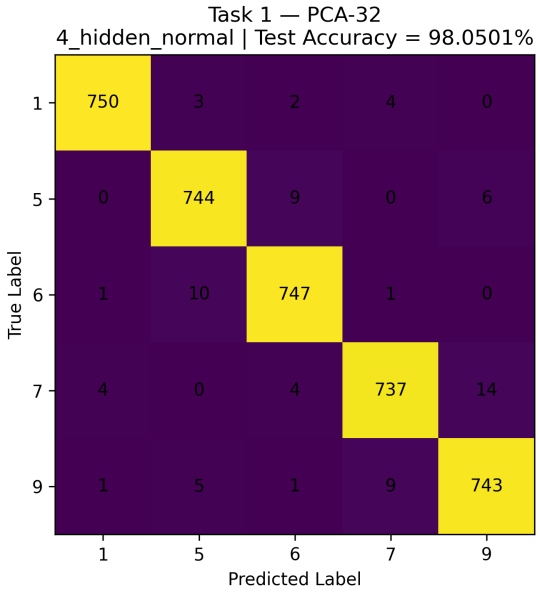

Fig. 1. Confusion matrix: confusion matrix pca for 32 components.

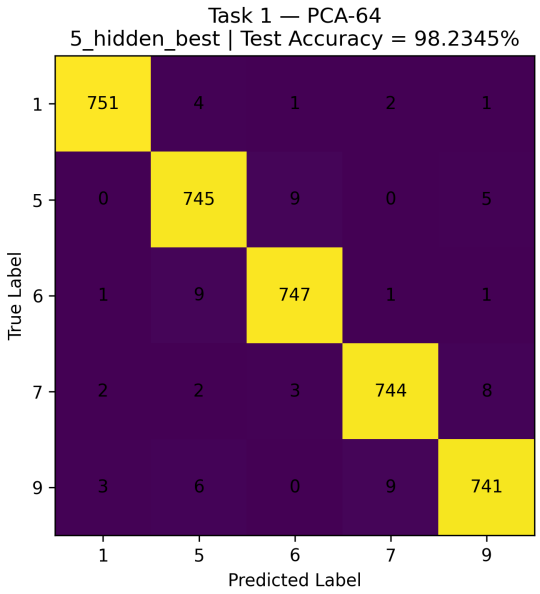

Fig. 2. Confusion matrix: confusion matrix pca for 64 components.

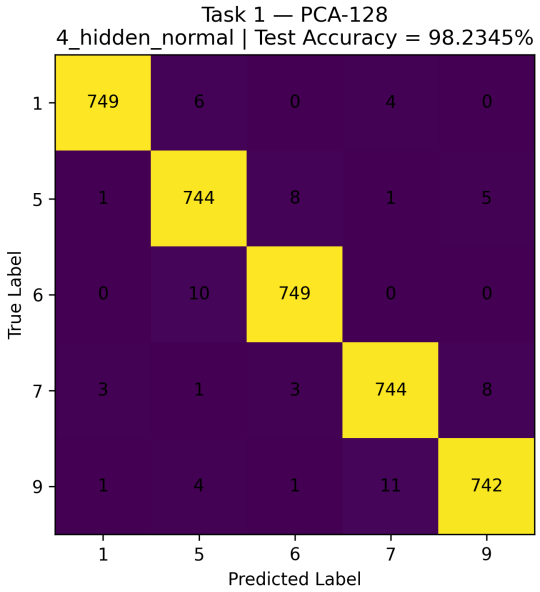

Fig. 3. Confusion matrix: confusion matrix pca for 128 components.

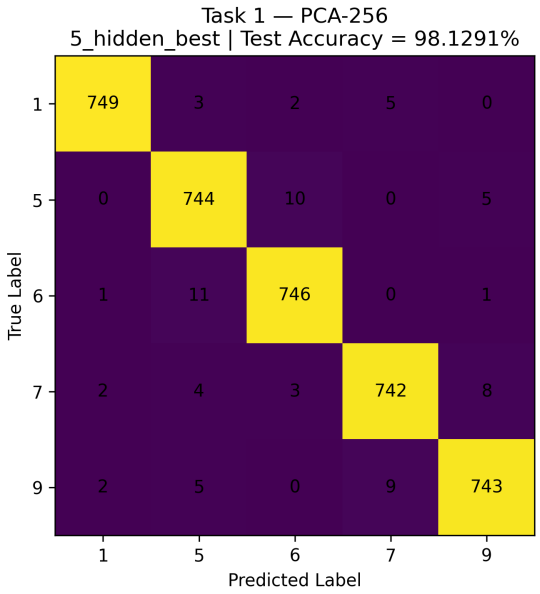

Fig. 4. Confusion matrix: confusion matrix pca for 256 components.

IV. Task 2: Autoencoder Reconstruction

A. *Architectures and training*

Two autoencoder(AE) families are evaluated at bottleneck dimensions 32, 64, 128, and 256. The one-hidden-layer model maps the 784-dimensional input directly to the bottleneck and then reconstructs the input. The three-hidden-layer model uses the architecture $784 \to 400 \to d \to 400 \to 784$, where d is the bottleneck dimension. The one-hidden-layer encoder and decoder use logistic activations except for the linear bottleneck; the decoder output also uses logistic so that reconstructed pixels remain in $[0, 1]$. In the three-hidden-layer model, both 400-unit hidden layers use logistic activation, the bottleneck is linear, and the output layer uses logistic. Both families are optimized with Adam at learning rate 0.001 for 500 epochs. The supplied final results correspond to the 500-epoch runs. Reconstruction error is evaluated separately on training, validation, and test data.

B. Quantitative results

Table III reports train, validation, and test reconstruction MSE for all eight trained autoencoders. For the one-hidden-layer autoencoder, test MSE decreases as the bottleneck grows from 32 to 256 dimensions. The three-hidden-layer model also improves between 32 and 256 dimensions, although its MSE is not lower than the one-hidden-layer model at every dimension. This illustrates that reconstruction quality depends on both bottleneck capacity and architecture rather than depth alone.

TABLE III
RECONSTRUCTION MSE ON TRAINING, VALIDATION, AND TEST SETS AFTER THE SELECTED 500-EPOCH RUNS.

Bottleneck	Architecture	Train MSE	Validation MSE	Test MSE
32	One-hidden AE	0.006452	0.007211	0.007314
64	One-hidden AE	0.002369	0.003019	0.003065
128	One-hidden AE	0.000953	0.001416	0.001415
256	One-hidden AE	0.000498	0.000805	0.000805
32	Three-hidden AE	0.002815	0.005041	0.005136
64	Three-hidden AE	0.001463	0.003398	0.003467
128	Three-hidden AE	0.000761	0.002209	0.002226
256	Three-hidden AE	0.000582	0.001576	0.001588

The following figures preserve the supplied reconstruction examples for the training, validation, and test splits. These visualizations allow the reconstructed strokes and class-specific distortions to be inspected alongside the numerical MSE values.

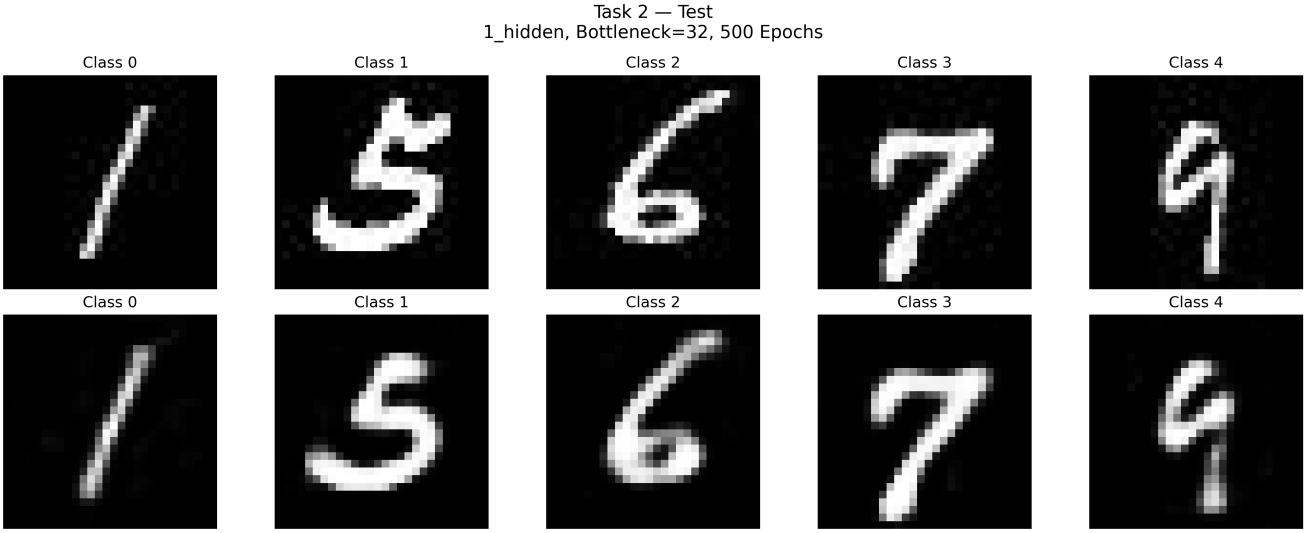

Fig. 5. Autoencoder reconstruction visualization: test 1 hidden with bottleneck of 32 neurons.

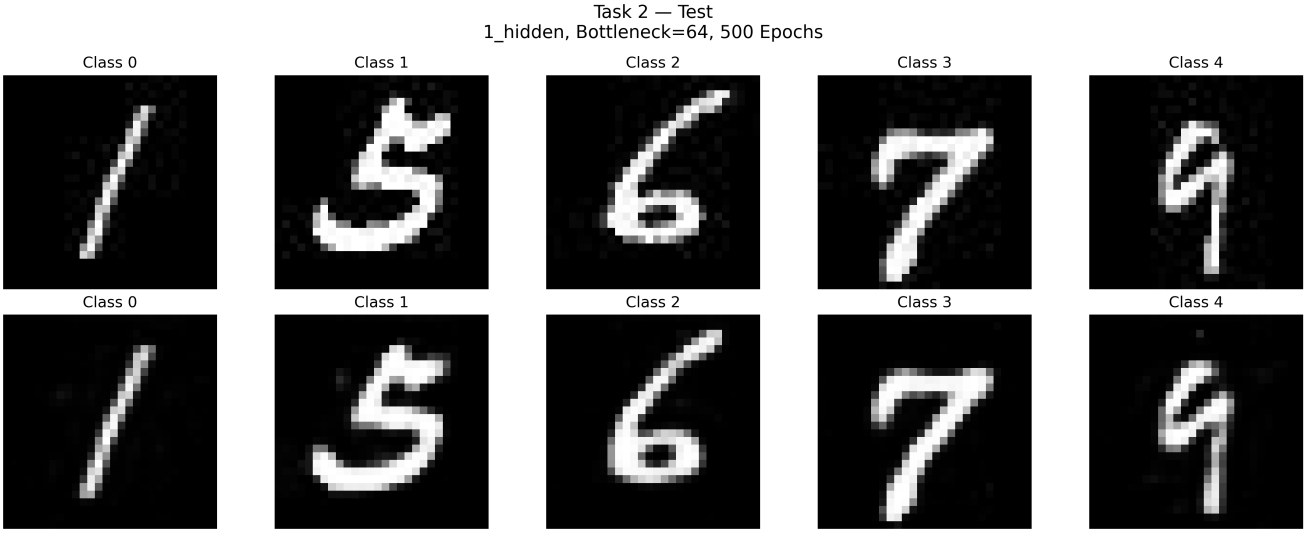

Fig. 6. Autoencoder reconstruction visualization: test 1 hidden with bottleneck of 64 neurons.

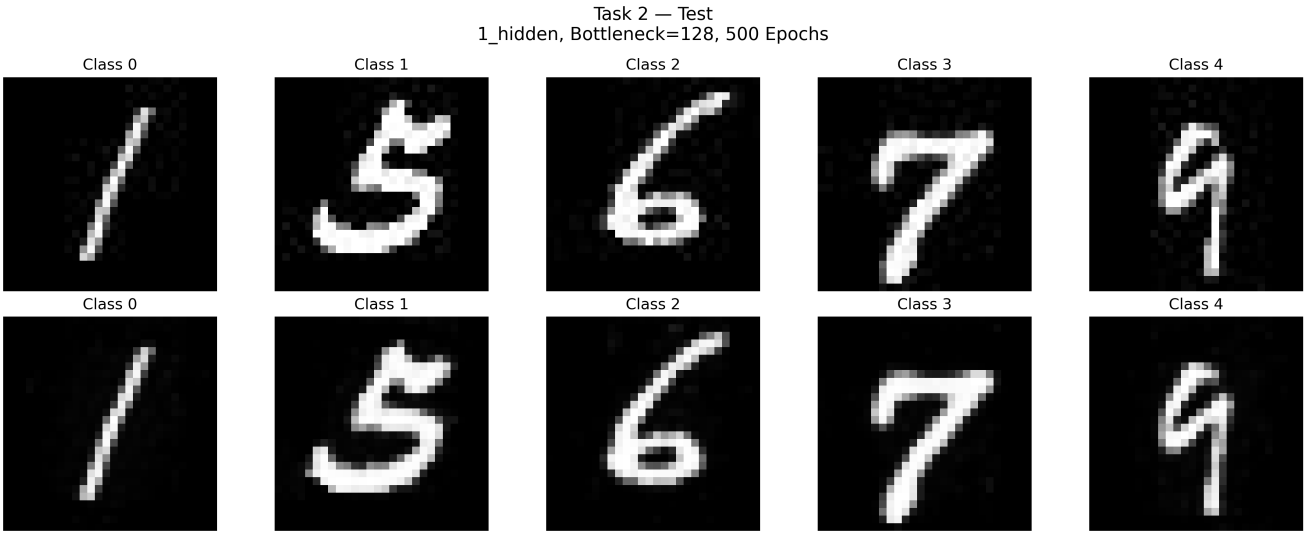

Fig. 7. Autoencoder reconstruction visualization: test 1 hidden with bottleneck of 128 neurons.

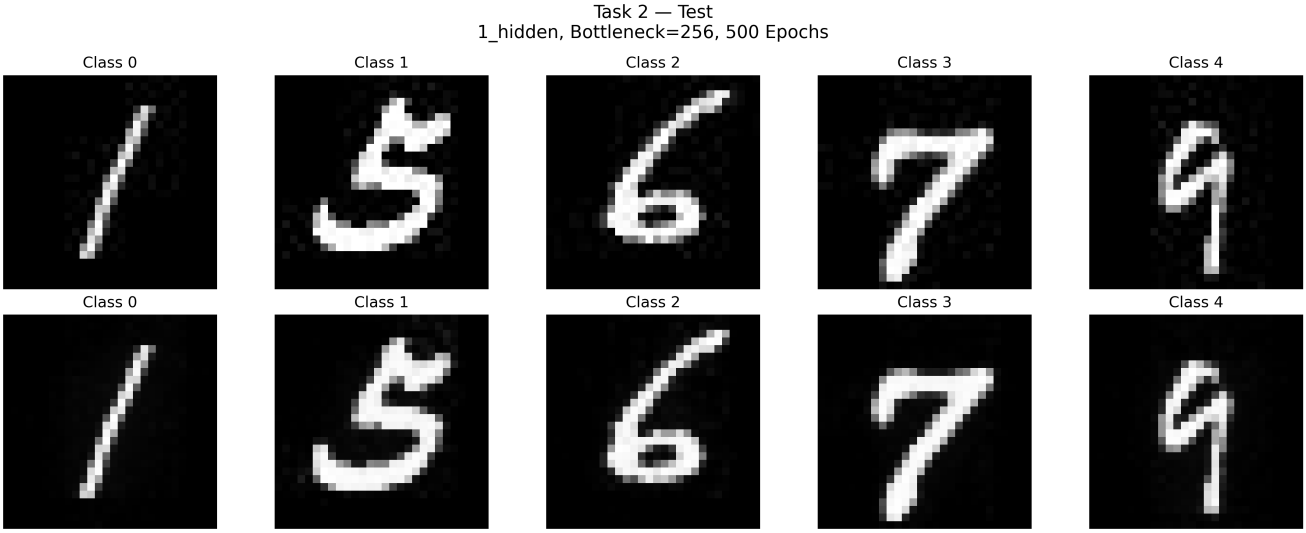

Fig. 8. Autoencoder reconstruction visualization: test 1 hidden with bottleneck of 256 neurons.

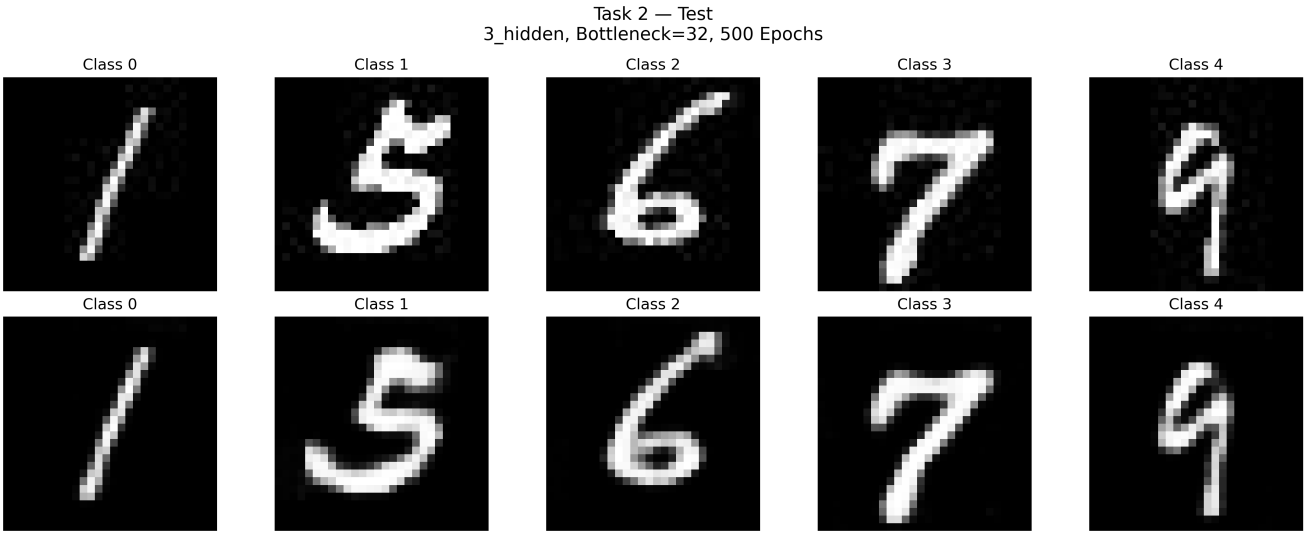

Fig. 9. Autoencoder reconstruction visualization: test 3 hidden with bottleneck of 32 neurons.

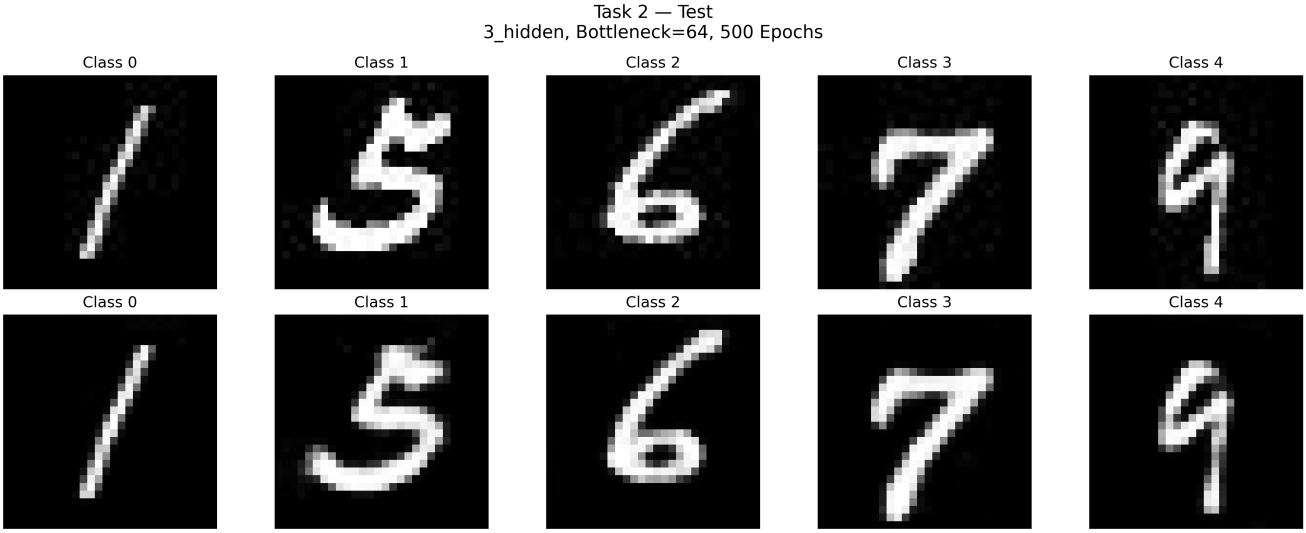

Fig. 10. Autoencoder reconstruction visualization: test 3 hidden with bottleneck of 64 neurons.

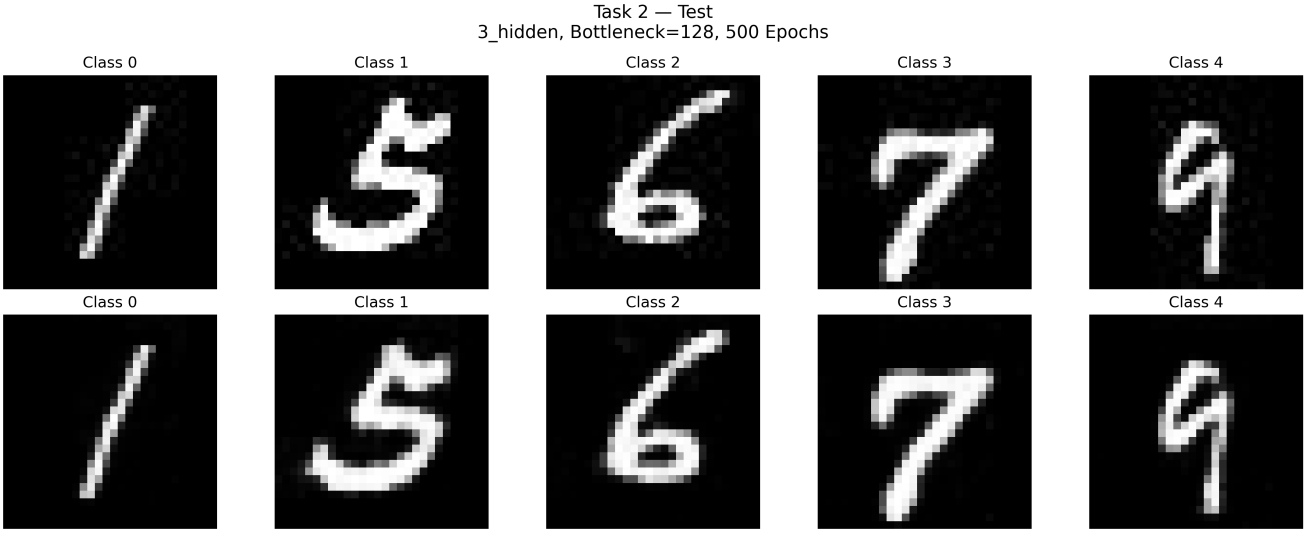

Fig. 11. Autoencoder reconstruction visualization: test 3 hidden with bottleneck of 128 neurons.

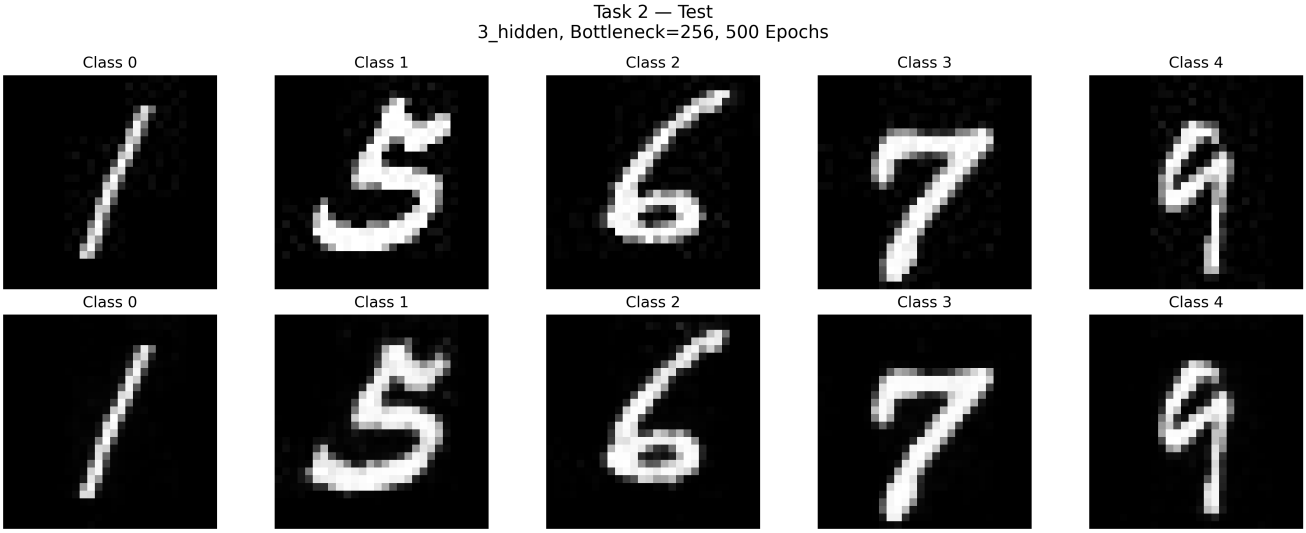

Fig. 12. Autoencoder reconstruction visualization: test 3 hidden with bottleneck of 256 neurons.

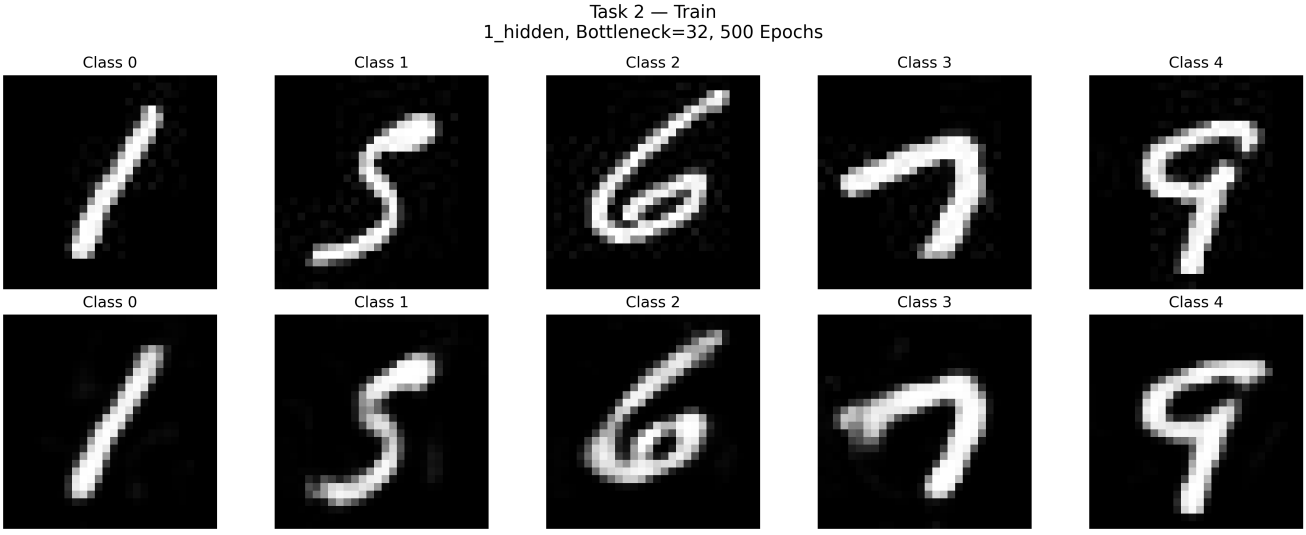

Fig. 13. Autoencoder reconstruction visualization: train 1 hidden with bottleneck of 32 neurons.

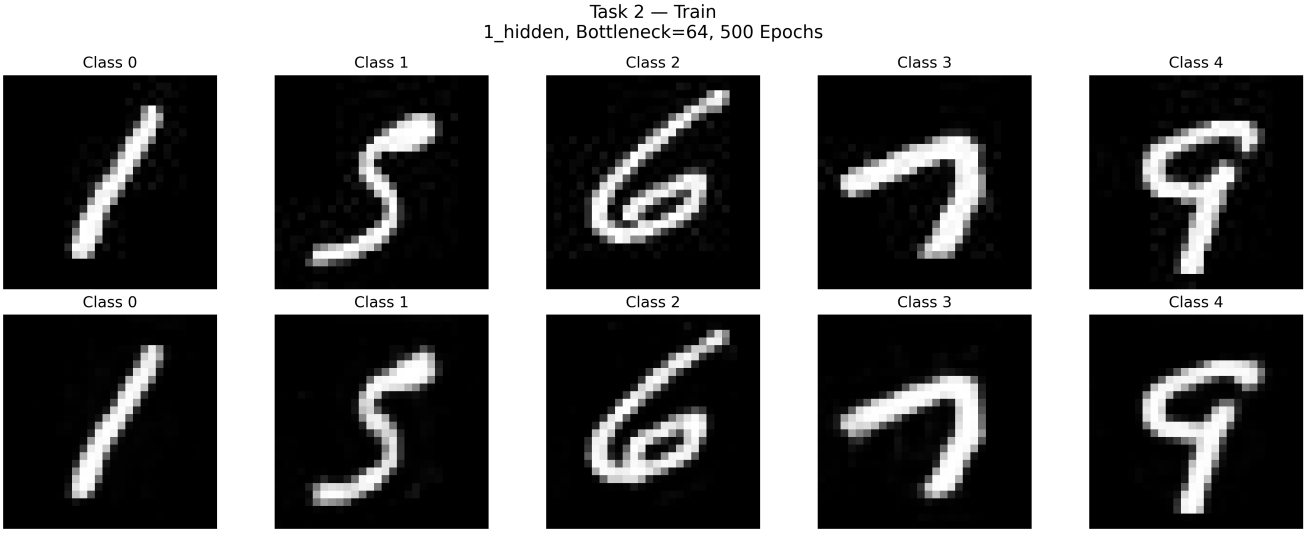

Fig. 14. Autoencoder reconstruction visualization: train 1 hidden with bottleneck of 64 neurons.

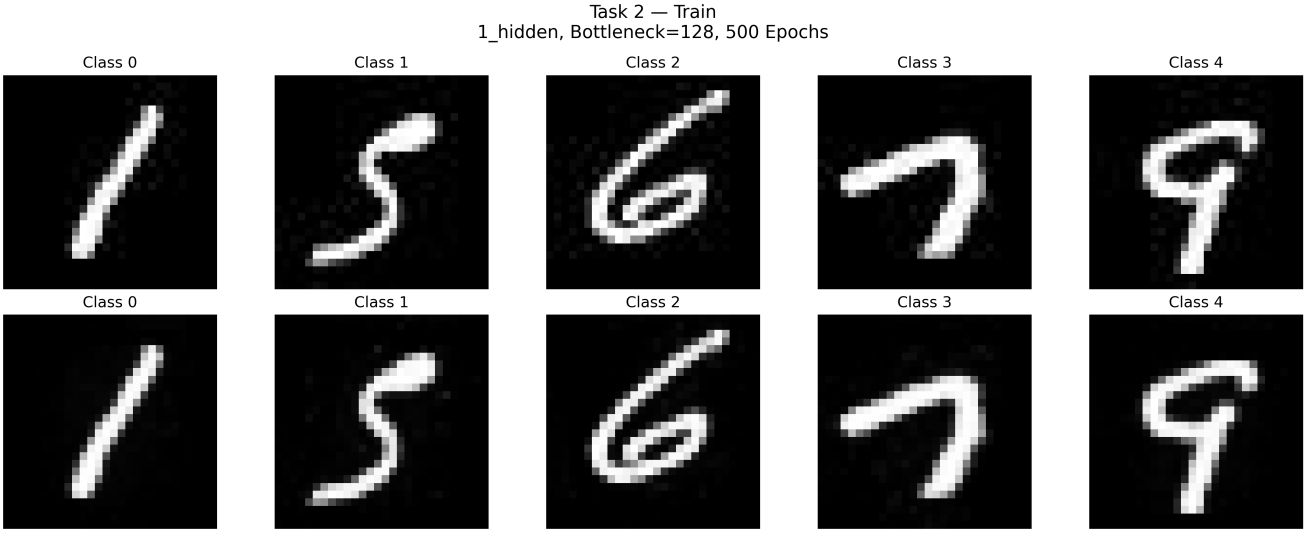

Fig. 15. Autoencoder reconstruction visualization: train 1 hidden with bottleneck of 128 neurons.

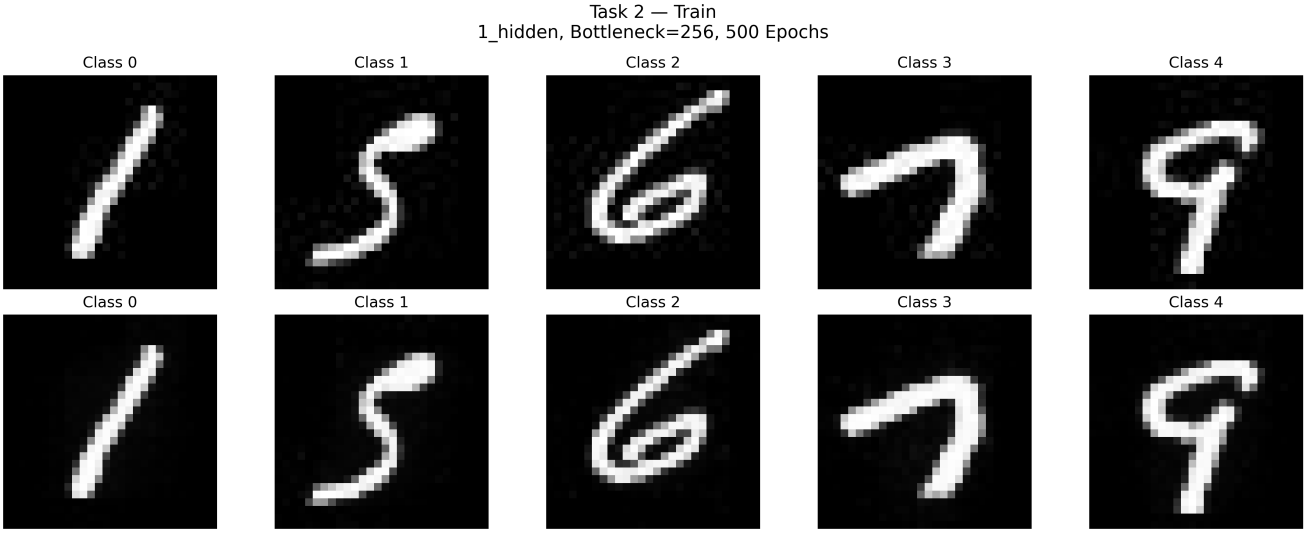

Fig. 16. Autoencoder reconstruction visualization: train 1 hidden with bottleneck of 256 neurons.

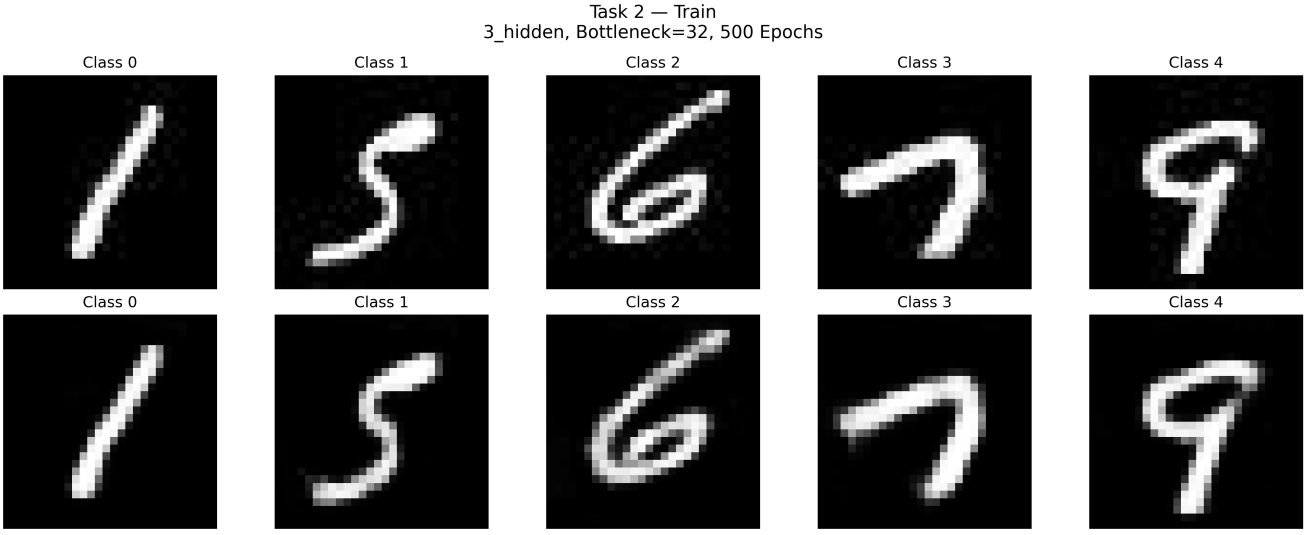

Fig. 17. Autoencoder reconstruction visualization: train 3 hidden with bottleneck of 32 neurons.

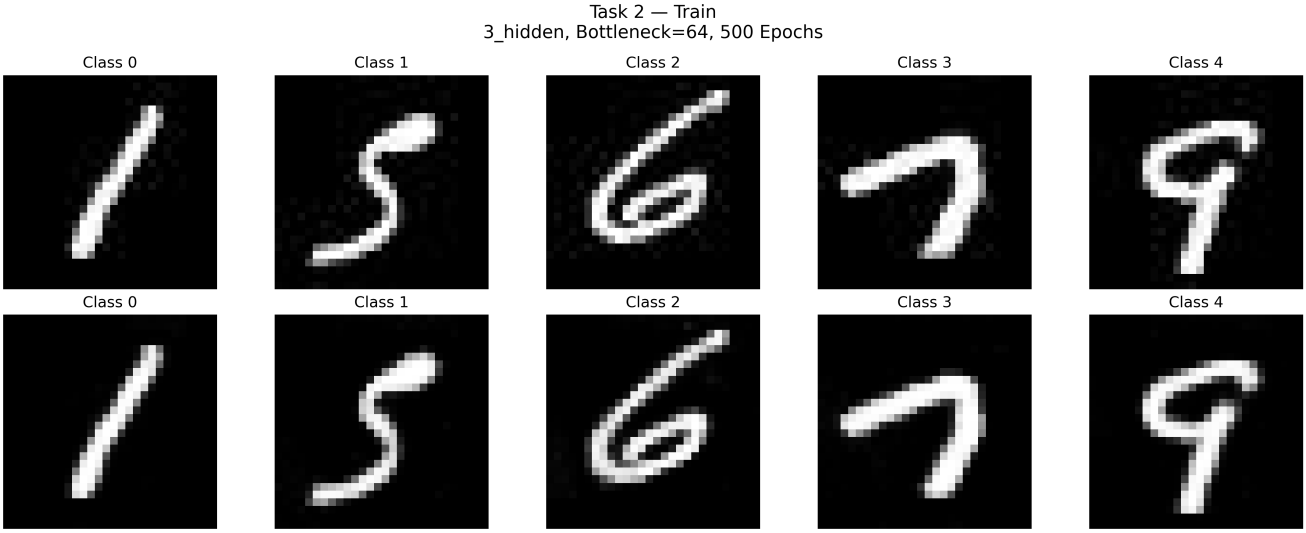

Fig. 18. Autoencoder reconstruction visualization: train 3 hidden with bottleneck of 64 neurons.

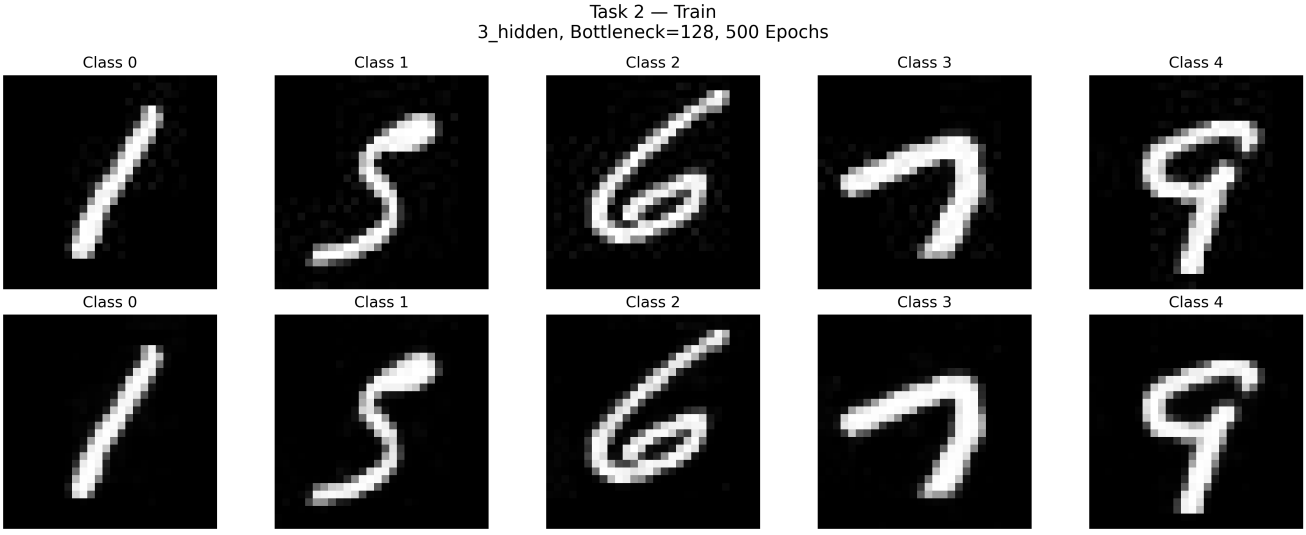

Fig. 19. Autoencoder reconstruction visualization: train 3 hidden with bottleneck of 128 neurons.

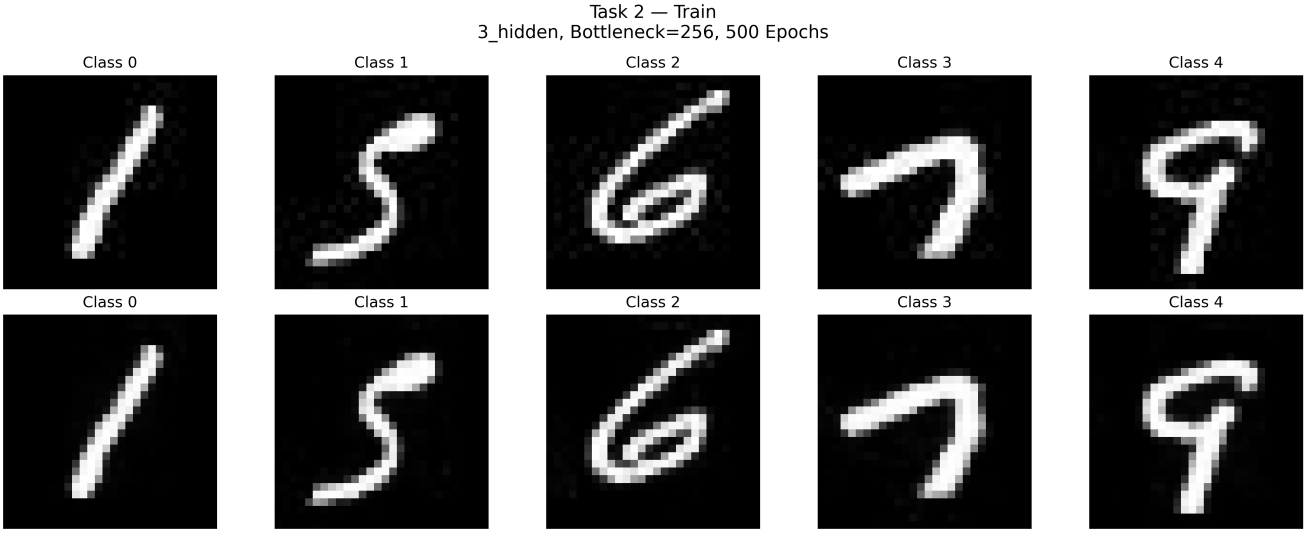

Fig. 20. Autoencoder reconstruction visualization: train 3 hidden with bottleneck of 256 neurons.

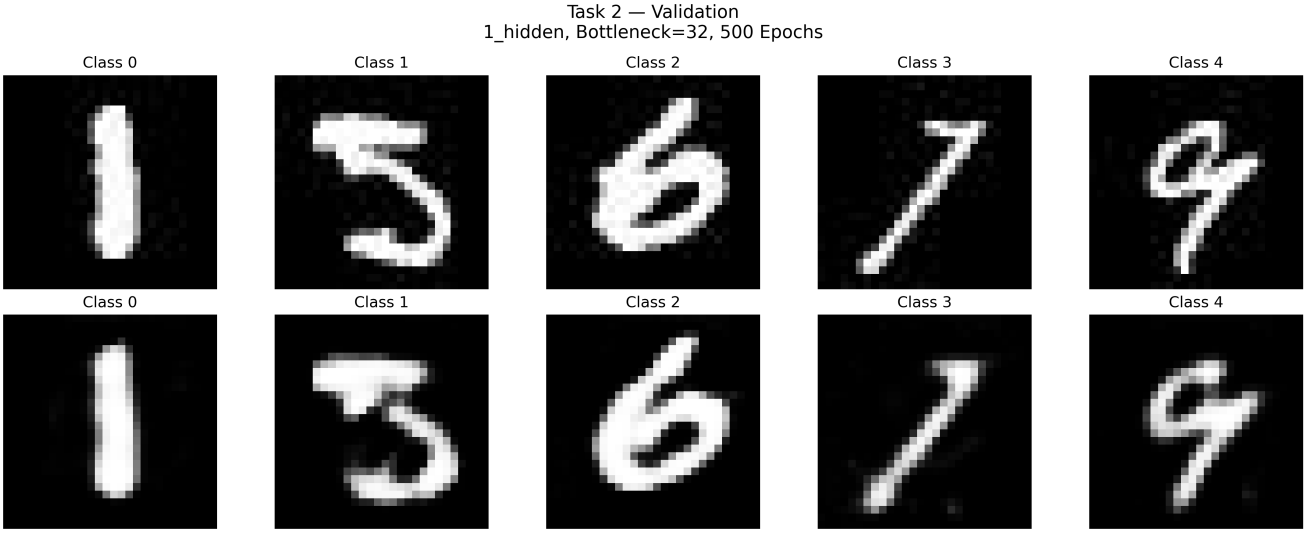

Fig. 21. Autoencoder reconstruction visualization: validation 1 hidden with bottleneck of 32 neurons.

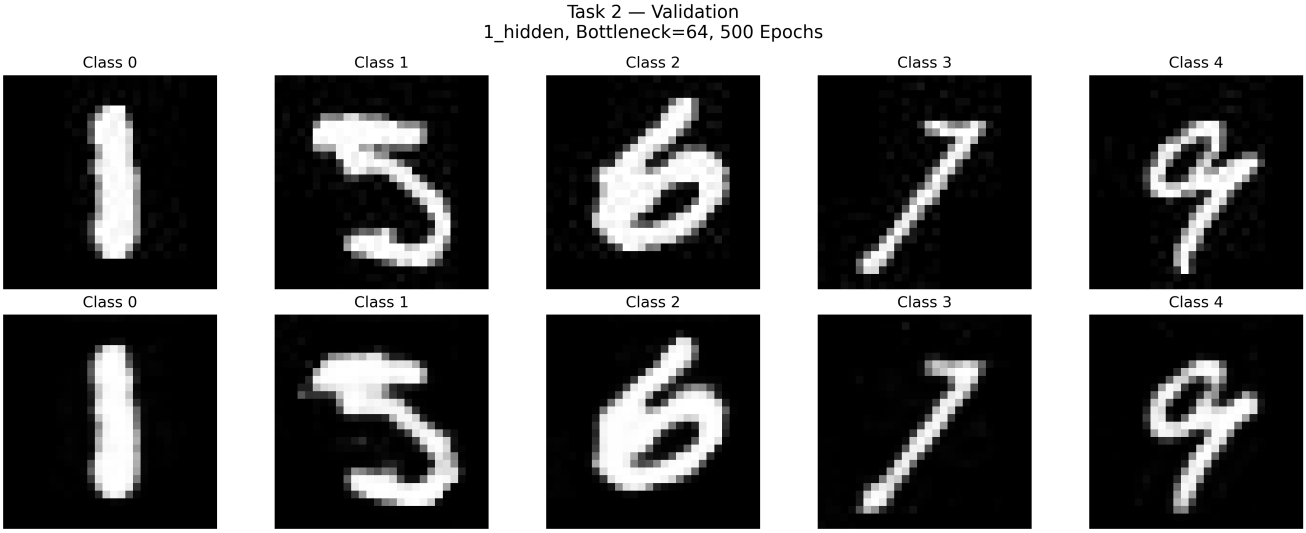

Fig. 22. Autoencoder reconstruction visualization: validation 1 hidden with bottleneck of 64 neurons.

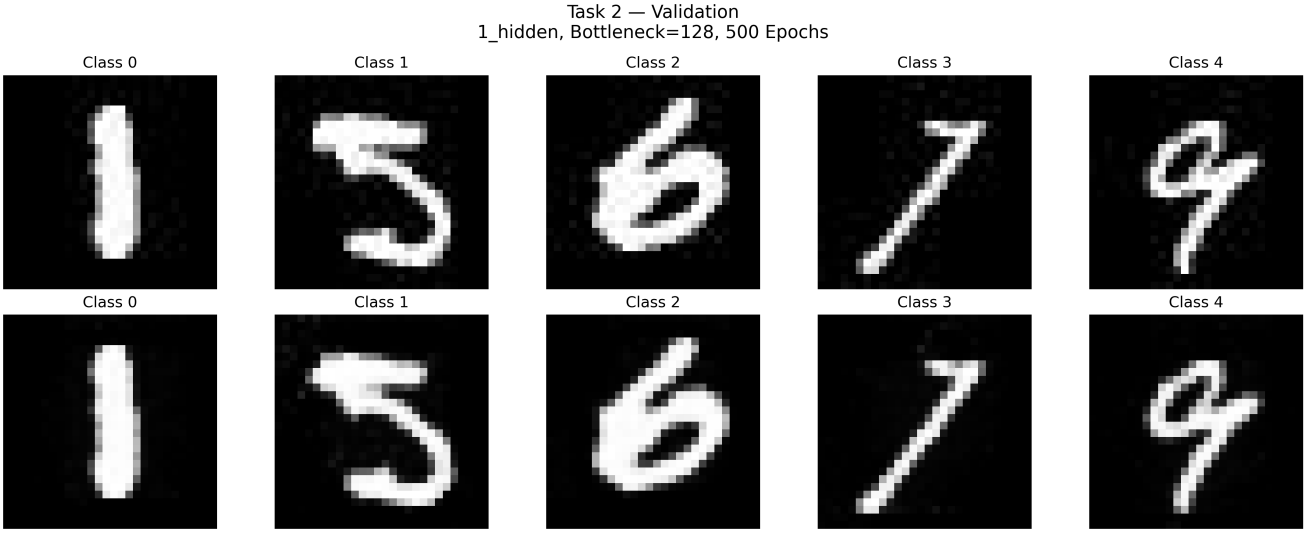

Fig. 23. Autoencoder reconstruction visualization: validation 1 hidden with bottleneck of 128 neurons.

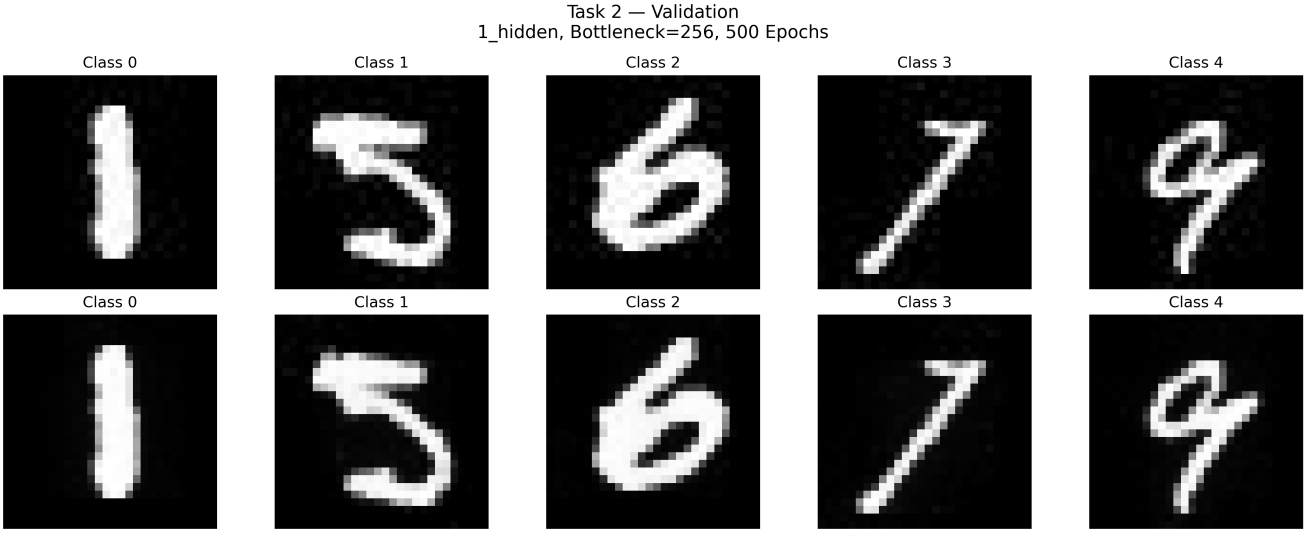

Fig. 24. Autoencoder reconstruction visualization: validation 1 hidden with bottleneck of 256 neurons.

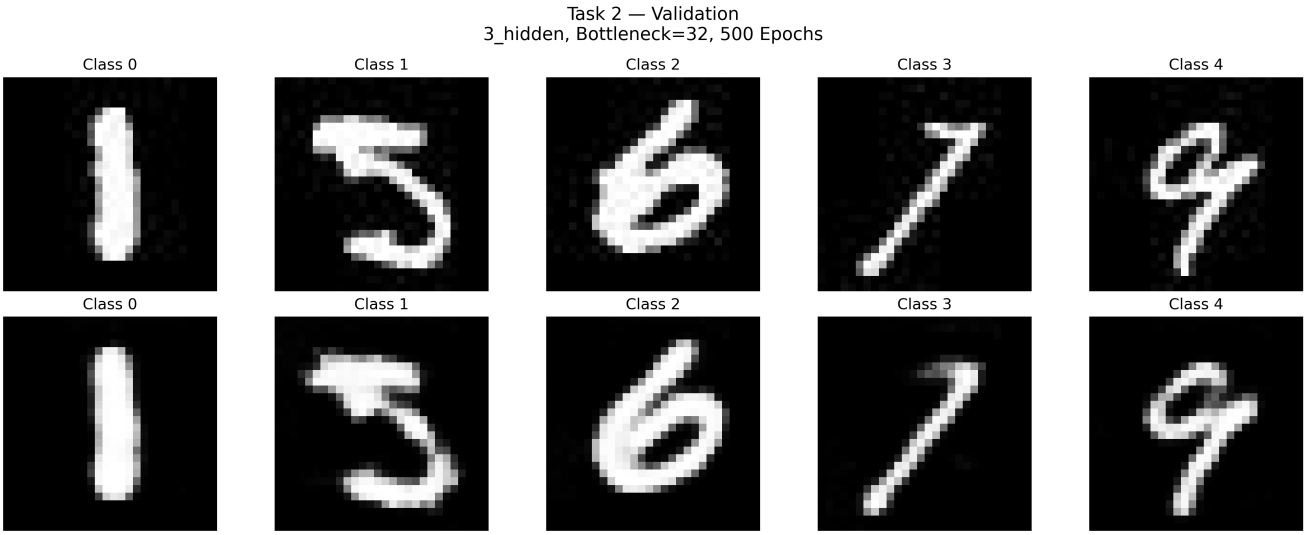

Fig. 25. Autoencoder reconstruction visualization: validation 3 hidden with bottleneck of 32 neurons.

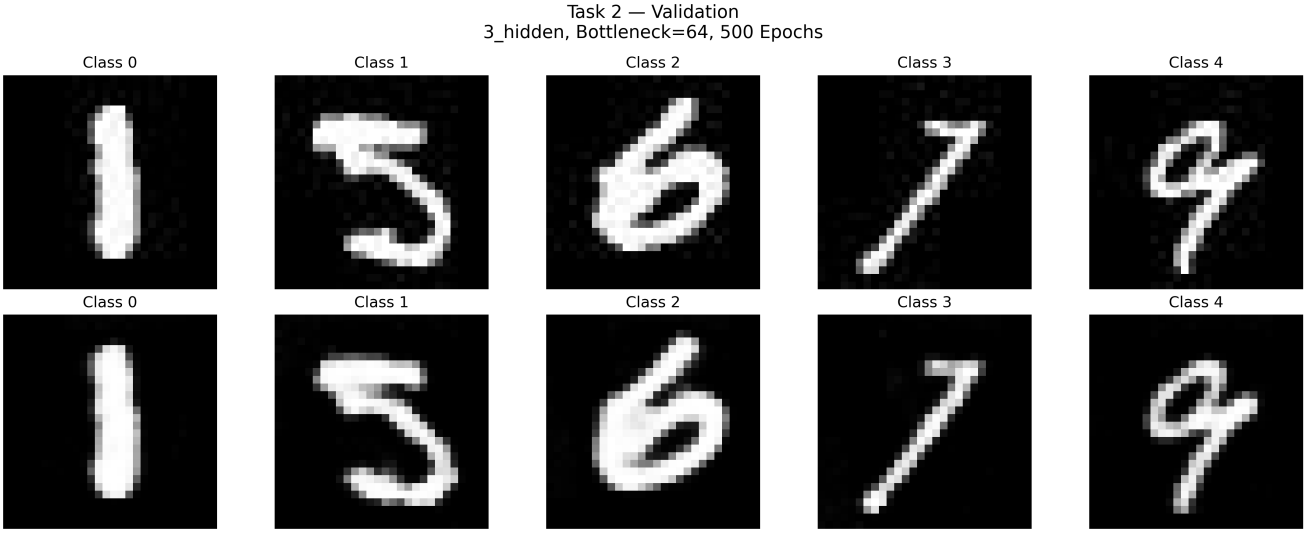

Fig. 26. Autoencoder reconstruction visualization: validation 3 hidden with bottleneck of 64 neurons.

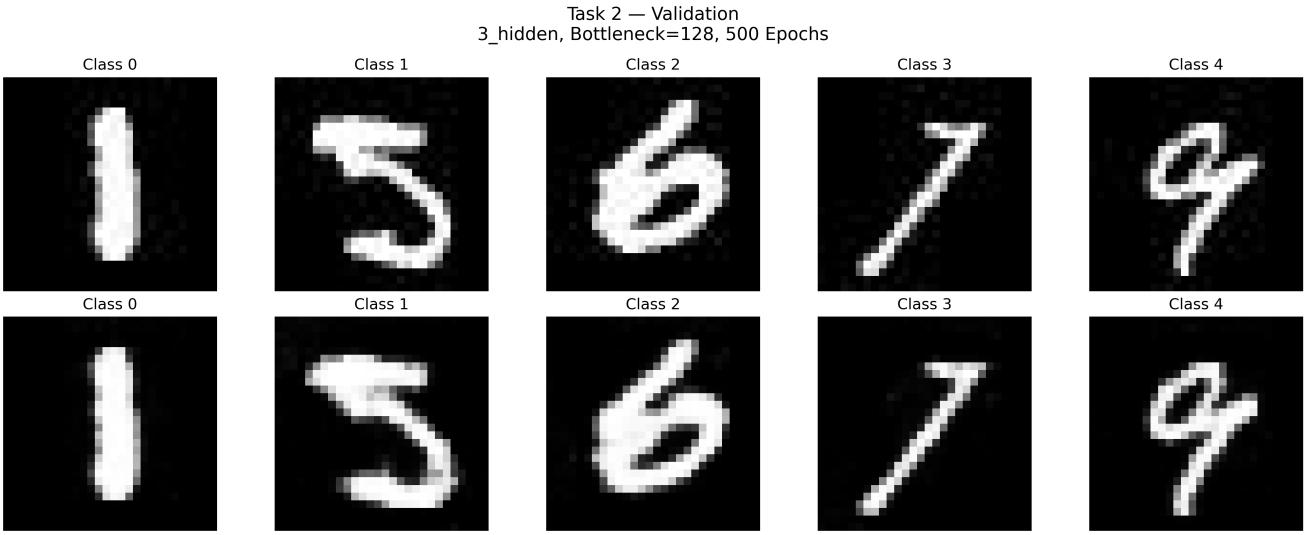

Fig. 27. Autoencoder reconstruction visualization: validation 3 hidden with bottleneck of 128 neurons.

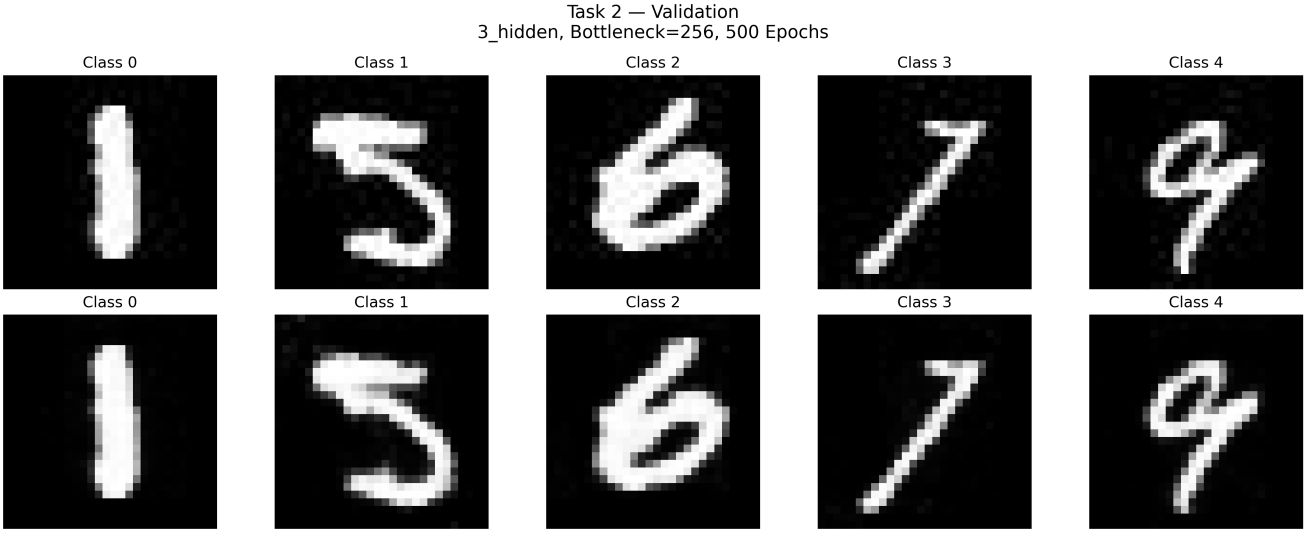

Fig. 28. Autoencoder reconstruction visualization: validation 3 hidden with bottleneck of 256 neurons.

V. Task 3: Classification Using One-Hidden-Layer Autoencoder Representations

A. Method

The encoder of each trained one-hidden-layer autoencoder is applied to the dataset to obtain 32-dimensional, 64-dimensional, 128-dimensional, or 256-dimensional features. The FCNN architectures are evaluated on these features, and validation accuracy is used to select the best architecture for each dimension.

B. Results

Table V shows that the 128-dimensional representation gives the highest test accuracy among the four dimensions, at 98.1555%. The 64-dimensional representation is close at 98.1028%, while the 32-dimensional and 256-dimensional representations are lower. Relative to Assignment 3 (98.1291%) and the best PCA result in Task 1 (98.2345%), the best one-hidden-layer AE representation is 0.0264 percentage points above the Assignment 3 baseline but 0.0790 percentage points below the PCA result. The AE representation is therefore useful, but it does not surpass the best PCA classifier in this experiment. This indicates that the representation with the largest number of latent features is not automatically the best for classification.

TABLE IV
Validation accuracy for every FCNN architecture and one-hidden-layer AE representation.

Representation dim.	FCNN architecture	Validation accuracy (%)
32	4_hidden_normal	98.6825
32	3_hidden_best	98.6298
32	4_hidden_best	98.3663
32	5_hidden_best	97.9974
64	4_hidden_best	98.5507
64	3_hidden_best	98.4453
64	4_hidden_normal	98.3663
64	5_hidden_best	98.3399
128	4_hidden_normal	98.5244
128	5_hidden_best	98.4453
128	4_hidden_best	98.3663
128	3_hidden_best	98.2609
256	5_hidden_best	98.4453
256	4_hidden_best	98.2082
256	4_hidden_normal	98.1291
256	3_hidden_best	98.0237

TABLE V
BEST CLASSIFICATION RESULT BY ONE-HIDDEN-LAYER AE REPRESENTATION.

Dim.	Architecture	Val. acc. (%)	Test acc. (%)
32	4-hidden normal	98.6825	97.9447
64	4-hidden best	98.5507	98.1028
128	4-hidden normal	98.5244	98.1555
256	5-hidden best	98.4453	97.9974

The confusion matrices provide additional detail on the errors made by the best classifier for each latent dimension.

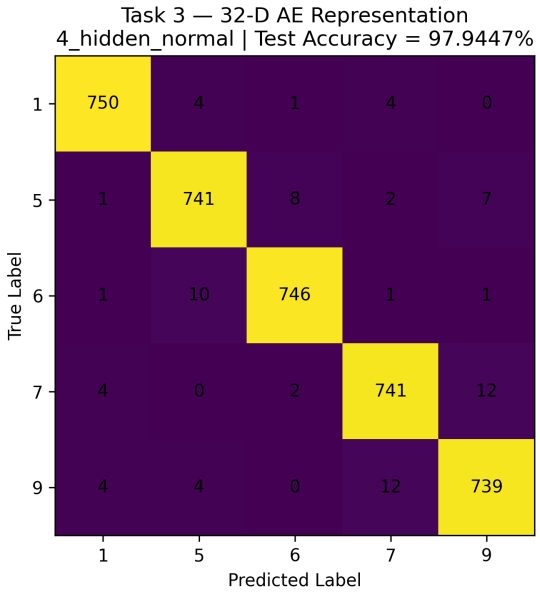

Fig. 29. Confusion matrix: confusion matrix for 32 neurons bottleneck.

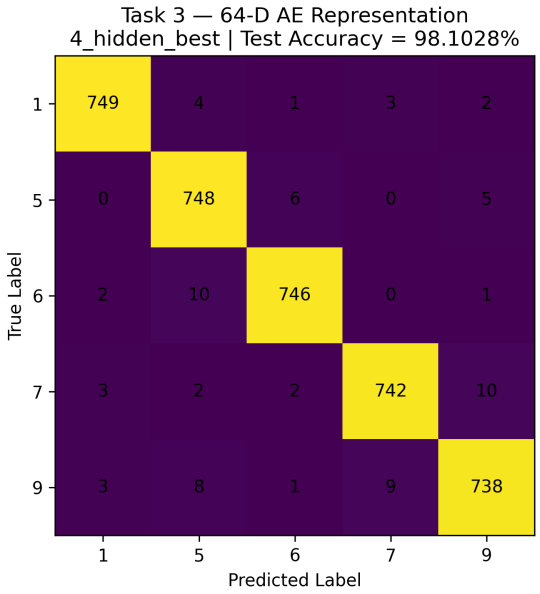

Fig. 30. Confusion matrix: confusion matrix for 64 neurons bottleneck.

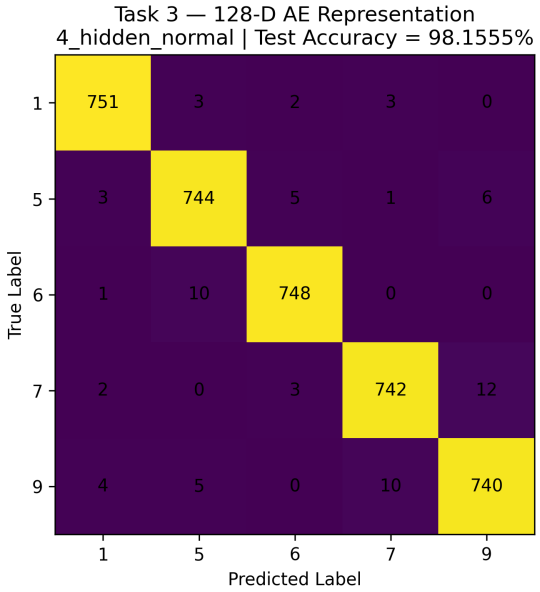

Fig. 31. Confusion matrix: confusion matrix for 128 neurons bottleneck.

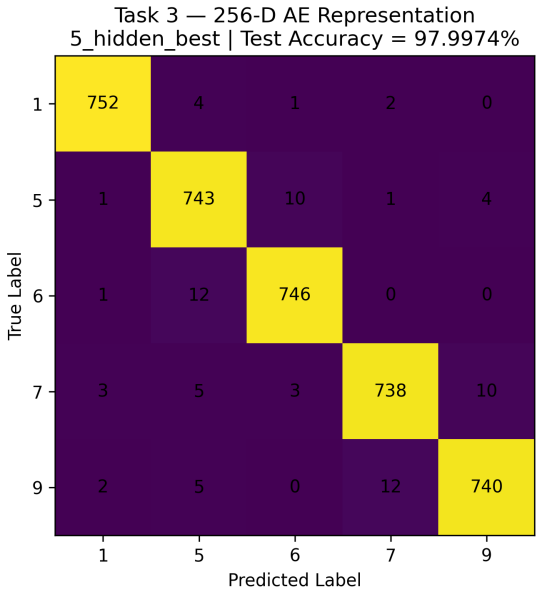

Fig. 32. Confusion matrix: confusion matrix for 256 neurons bottleneck.

VI. Task 4: Classification Using Three-Hidden-Layer Autoencoder Representations

A. *Encoder representation*

For this task, the encoder portion of the three-hidden-layer autoencoder is used up to and including the bottleneck. The resulting representations have dimensions 32, 64, 128, and 256. The same set of FCNN architectures and validation-based selection procedure is applied to each representation.

B. *Results*

The 64-dimensional representation produces the best test accuracy, 98.2872%, using the five-hidden-layer FCNN configuration. The 32-dimensional representation reaches 98.2345%, whereas the 128- and 256-dimensional representations are lower. Compared with Assignment 3 (98.1291%), Task 1 PCA (98.2345%), and Task 3 (one-hidden-layer AE, 98.1555%), the 64-D three-hidden-layer AE encoder is best among these four methods at 98.2872%. It exceeds the

Assignment 3 baseline by 0.1581 percentage points, PCA by 0.0527 percentage points, and Task 3 by 0.1317 percentage points. The results again show that the best representation dimension is determined empirically and need not be the largest.

TABLE VI
VALIDATION ACCURACY FOR EVERY FCNN ARCHITECTURE AND THREE-HIDDEN-LAYER AE REPRESENTATION.

Representation dim.	FCNN architecture	Validation accuracy (%)
32	3_hidden_best	98.6034
32	4_hidden_best	98.5507
32	4_hidden_normal	98.5507
32	5_hidden_best	98.7615
64	3_hidden_best	98.4190
64	4_hidden_best	98.3399
64	4_hidden_normal	98.3663
64	5_hidden_best	98.4453
128	3_hidden_best	98.1555
128	4_hidden_best	98.3136
128	4_hidden_normal	98.4453
128	5_hidden_best	98.3136
256	3_hidden_best	98.1028
256	4_hidden_best	98.4717
256	4_hidden_normal	98.2082
256	5_hidden_best	98.2345

TABLE VII
BEST CLASSIFICATION RESULT BY THREE-HIDDEN-LAYER AE REPRESENTATION.

Dim.	Architecture	Val. acc. (%)	Test acc. (%)
32	5-hidden best	98.7615	98.2345
64	5-hidden best	98.4453	98.2872
128	4-hidden normal	98.4453	97.8393
256	4-hidden best	98.4717	97.9183

The confusion matrices for the four dimensions are shown next.

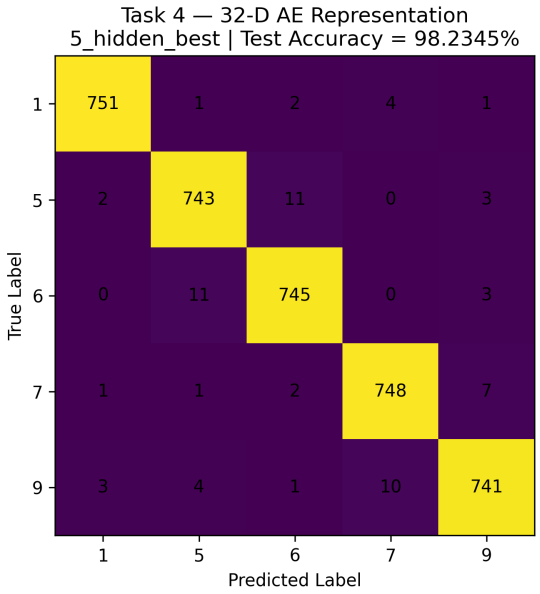

Fig. 33. Confusion matrix: confusion matrix for 32 neurons bottleneck.

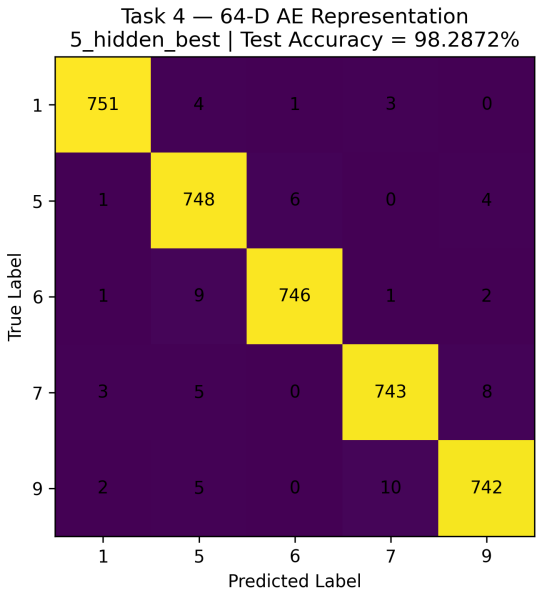

Fig. 34. Confusion matrix: confusion matrix for 64 neurons bottleneck.

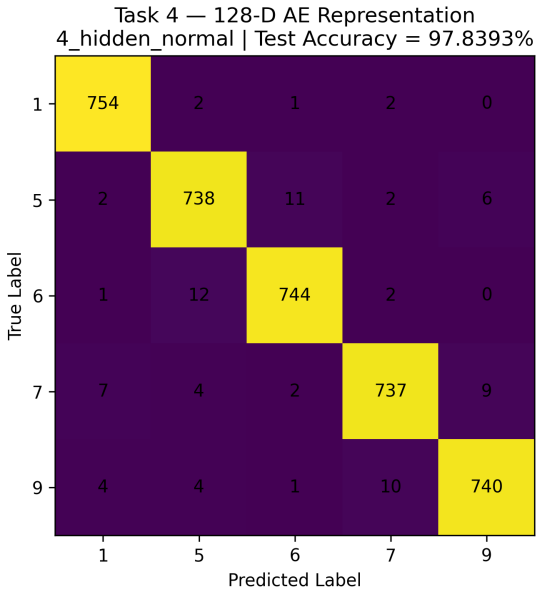

Fig. 35. Confusion matrix: confusion matrix for 128 neurons bottleneck.

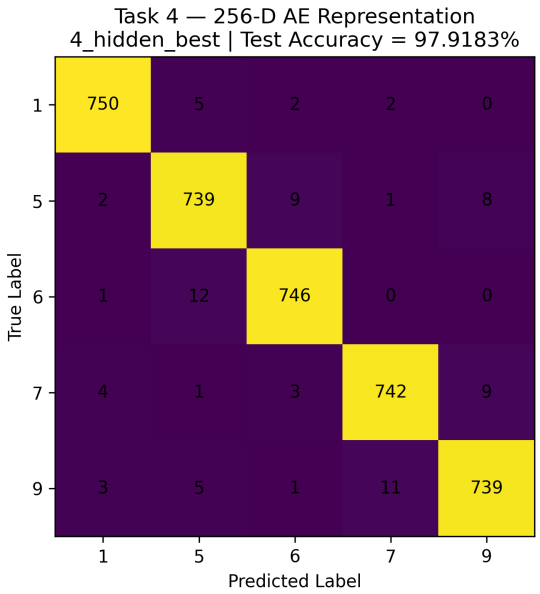

Fig. 36. Confusion matrix: confusion matrix for 256 neurons bottleneck.

VII. Task 5: Denoising Autoencoders

A. Noise model and evaluation

One-hidden-layer denoising autoencoders (DAEs) are evaluated using 20% and 40% corruption, with the bottleneck dimension fixed at 128. Gaussian noise is sampled from a standard normal distribution, $\epsilon \sim \mathcal{N}(0,1)$. A Bernoulli mask $m_i \sim \text{Bernoulli}(q)$ selects pixels to corrupt, where $q \in \{0.20, 0.40\}$. The noisy input used in the notebook is

$$\tilde{x}_i = x_i + m_i \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0,1), \quad m_i \sim \text{Bernoulli}(q). \tag{15}$$

This corresponds to the code that samples a random mask and uses `torch.randn_like` for the $N(0,1)$ noise. The target remains the clean input x. The autoencoders are trained to reconstruct clean inputs from corrupted inputs. Both reconstruction MSE and downstream classification accuracy are reported. The denoising autoencoders are trained for up to 500 epochs with Adam (learning rate 0.001, batch size 128). Their downstream FCNN uses the best Task 3 architecture for the 128-D representation, namely `4_hidden_normal` with hidden widths $[256, 128, 64, 32]$, trained with SGD with momentum ($\eta = 0.001$, $\mu = 0.9$, batch size 1) and the same FCNN convergence criterion.

B. Reconstruction results

Table VIII shows that the 20% noise model has lower reconstruction MSE than the 40% noise model on all three splits. The increase in error with stronger corruption is consistent with the more difficult reconstruction problem.

TABLE VIII
Task 5: Denoising reconstruction MSE.

Noise level	Train MSE	Val. MSE	Test MSE
20%	0.010297	0.010644	0.010699
40%	0.015991	0.016365	0.016454

C. Classification results

Both denoising representations are classified using the 128-dimensional feature setup and the `4_hidden_normal` architecture. The 20% DAE achieves 98.4453% test accuracy, and the 40% DAE achieves 98.4190%. The 20% DAE therefore gives the stronger classification result in this comparison. Notably, reconstruction MSE and classification accuracy measure different objectives: the model with lower reconstruction error is not necessarily the model with the best downstream classification result across all tasks.

TABLE IX
DAE CLASSIFICATION RESULTS.

Noise level	Val. acc. (%)	Test acc. (%)
20%	98.5771	98.4453
40%	98.5507	98.4190

The following figures include the classification confusion matrices and reconstruction examples for both noise levels and all supplied splits.

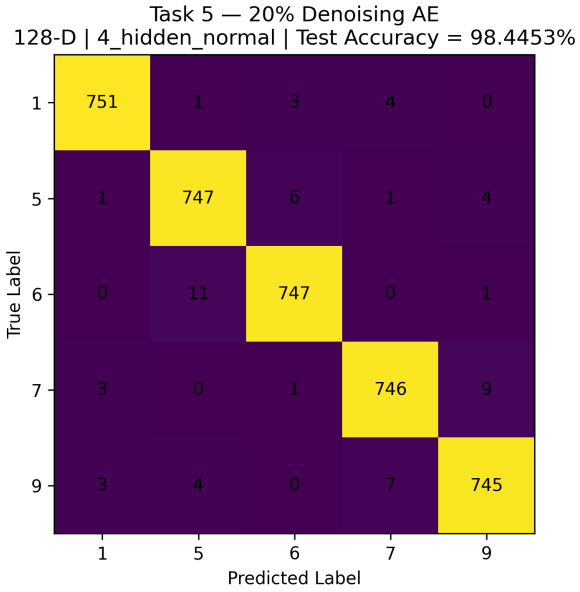

Fig. 37. Confusion matrix: confusion matrix for 20% noise.

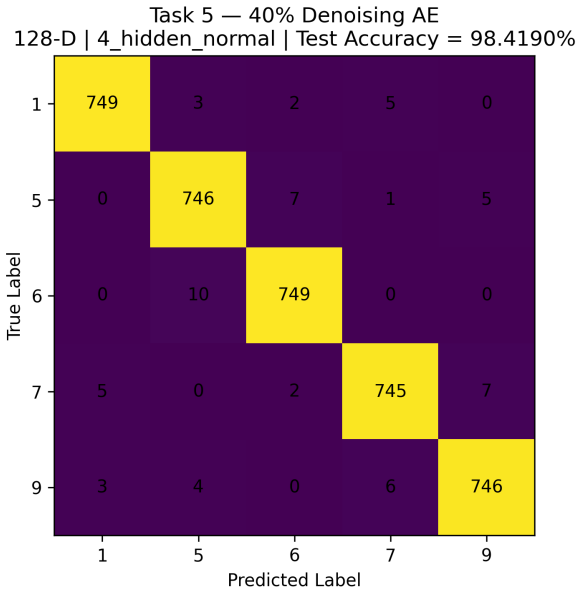

Fig. 38. Confusion matrix: confusion matrix for 40% noise.

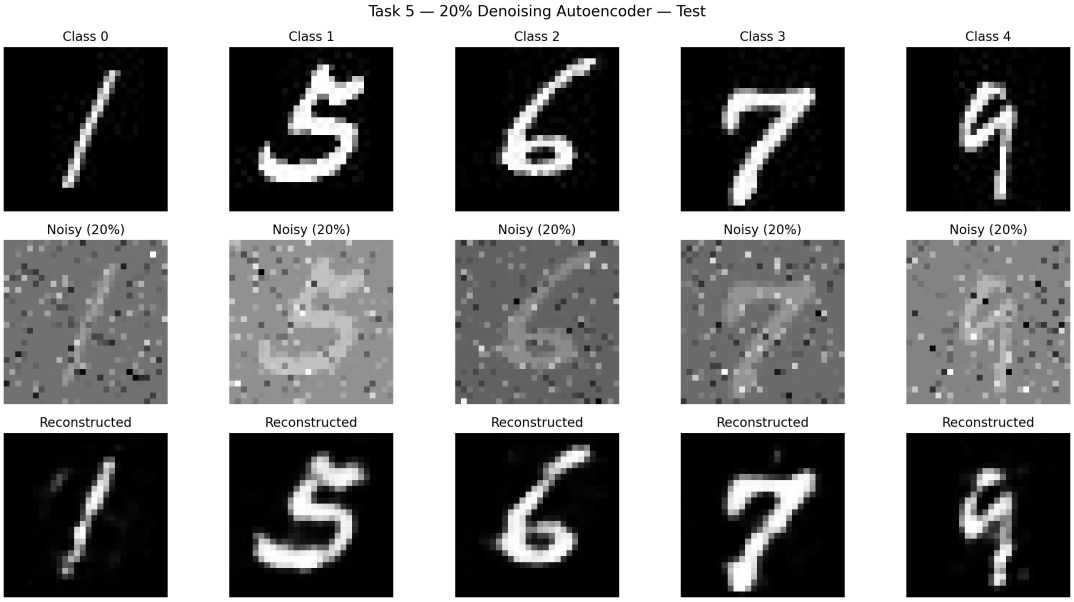

Fig. 39. Autoencoder reconstruction visualization: dae 20% noise test reconstructions.

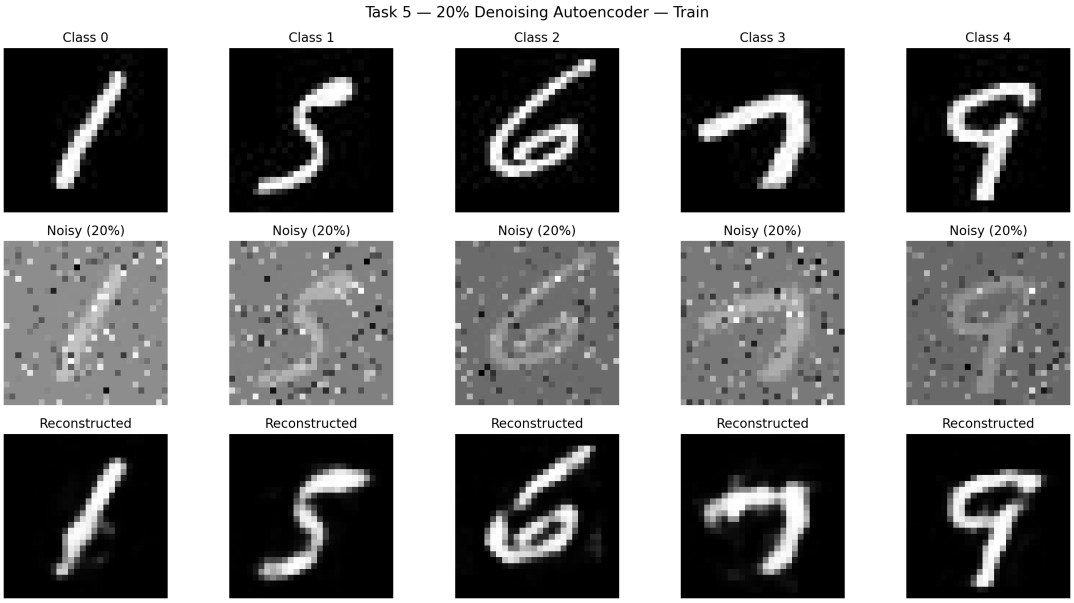

Fig. 40. Autoencoder reconstruction visualization: dae 20% noise train reconstructions.

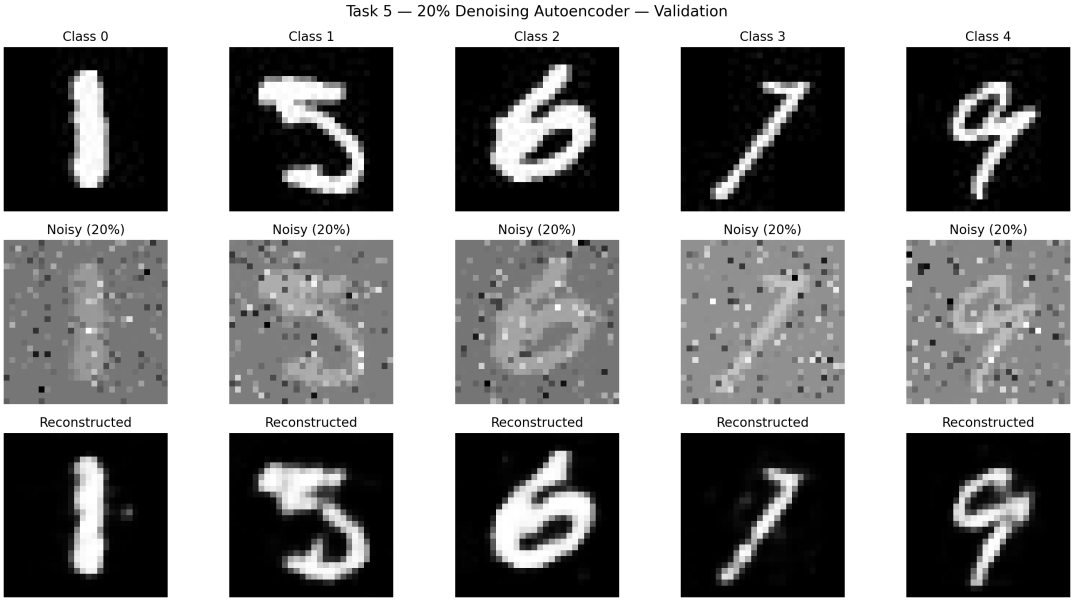

Fig. 41. Autoencoder reconstruction visualization: dae 20% noise validation reconstructions.

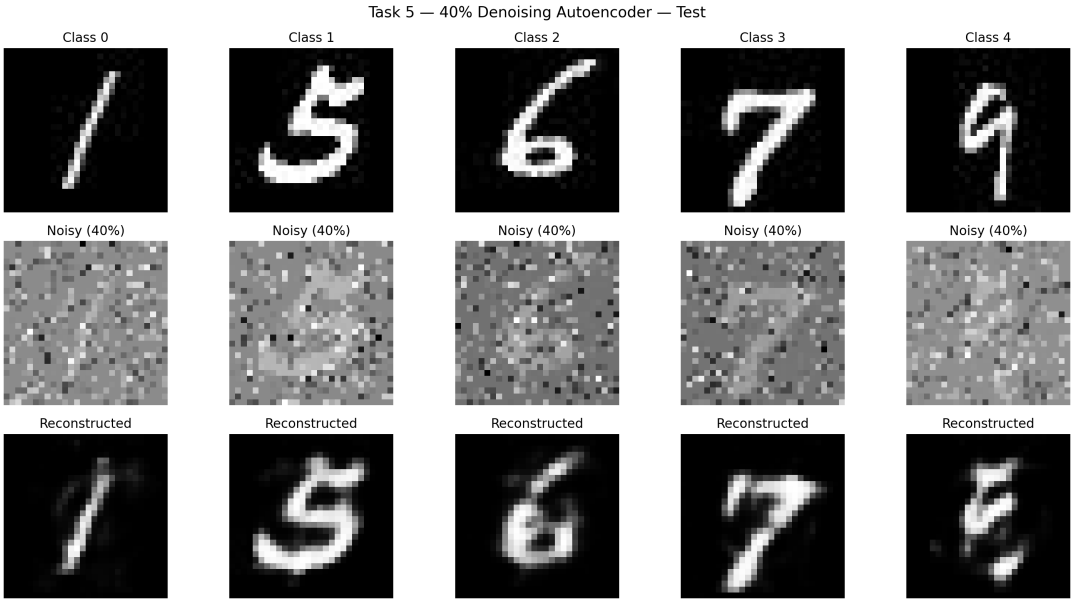

Fig. 42. Autoencoder reconstruction visualization: dae 40% noise test reconstructions.

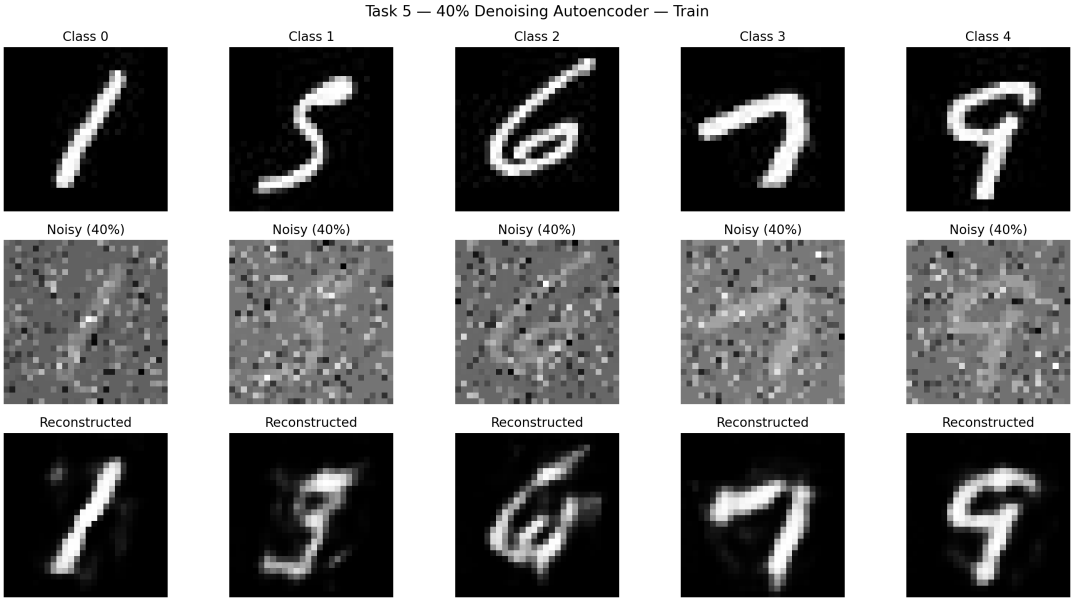

Fig. 43. Autoencoder reconstruction visualization: dae 40% noise train reconstructions.

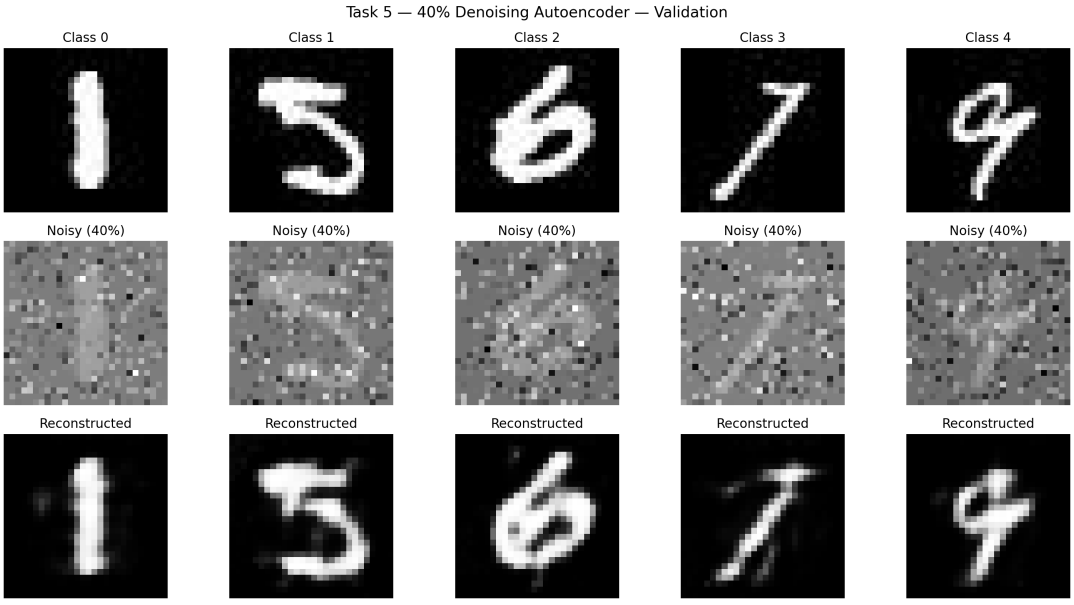

Fig. 44. Autoencoder reconstruction visualization: dae 40% noise validation reconstructions.

VIII. Task 6: Input-to-Bottleneck Weight Visualization

A. Method

For the best 128-dimensional one-hidden-layer autoencoder representation and the two 128-dimensional denoising autoencoders, each bottleneck neuron has 784 incoming weights. Each vector is reshaped into a 28×28 map so that the spatial arrangement corresponds to the original image coordinates. The color map encodes signed weight values; the weights themselves are not modified by the visualization. Eight pages of 16 neurons are provided for each model, giving 24 figures in total.

B. Interpretation

The weight maps are diagnostic visualizations of learned connections, not reconstructed digit examples. Positive and negative colors indicate the sign of the weights, while spatial patterns show which input locations have stronger

connections to each bottleneck unit. A comparison should focus on differences in spatial organization and contrast across the conventional AE and the two DAEs; these maps alone do not establish semantic meaning for a neuron.

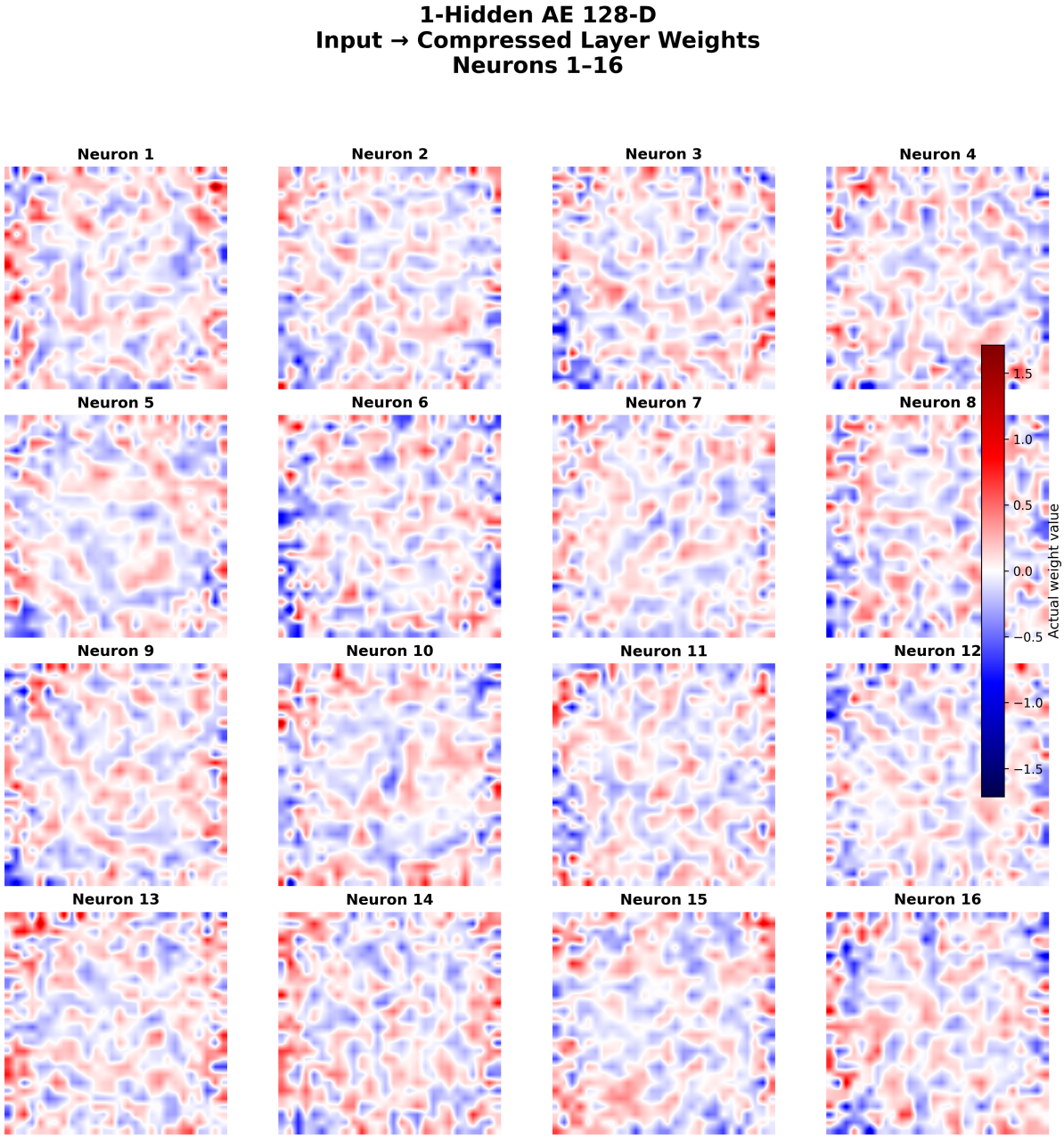

Fig. 45. Input-to-bottleneck weight visualization for neurons 1–16.

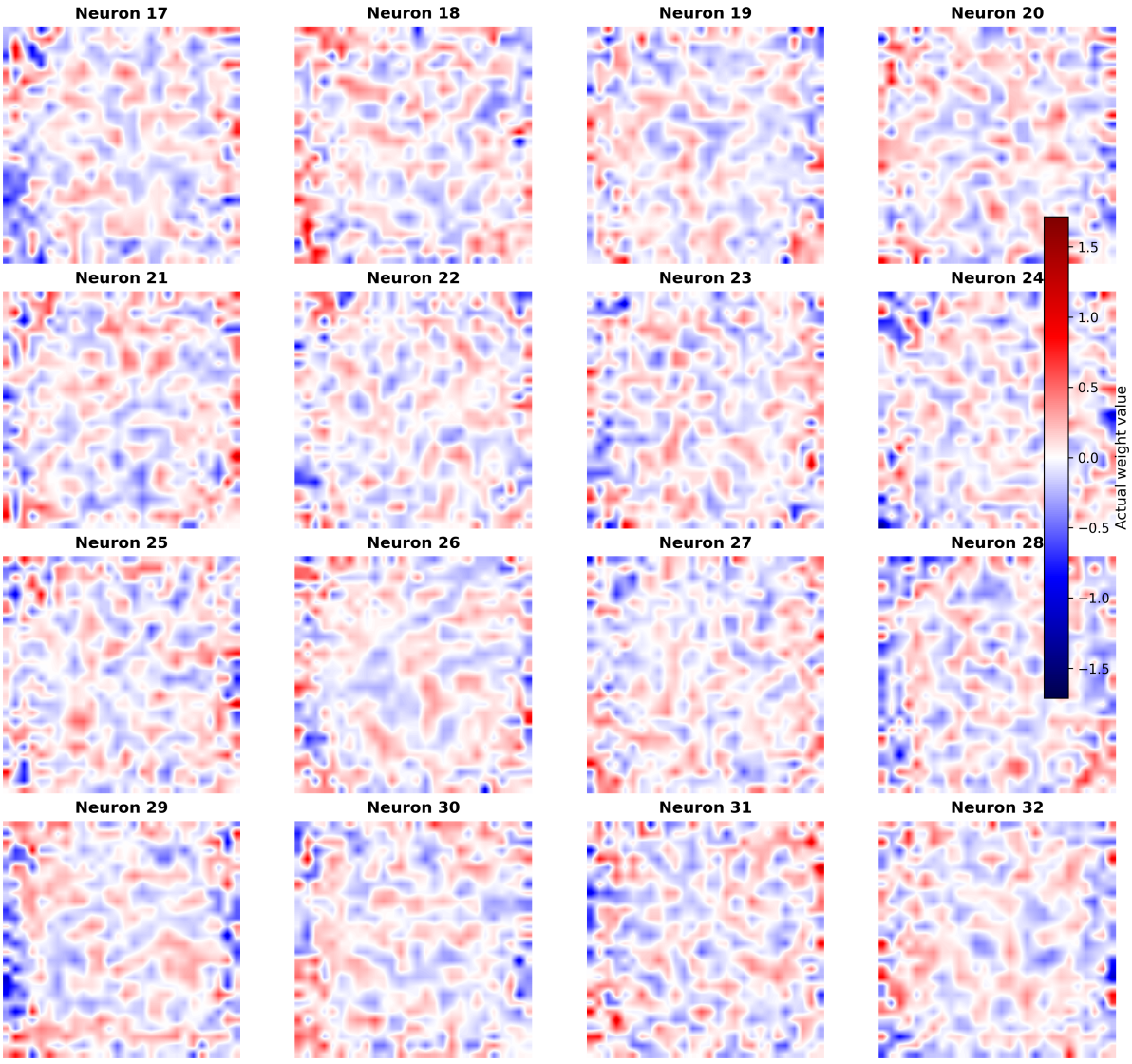

Fig. 46. Input-to-bottleneck weight visualization for neurons 17–32.

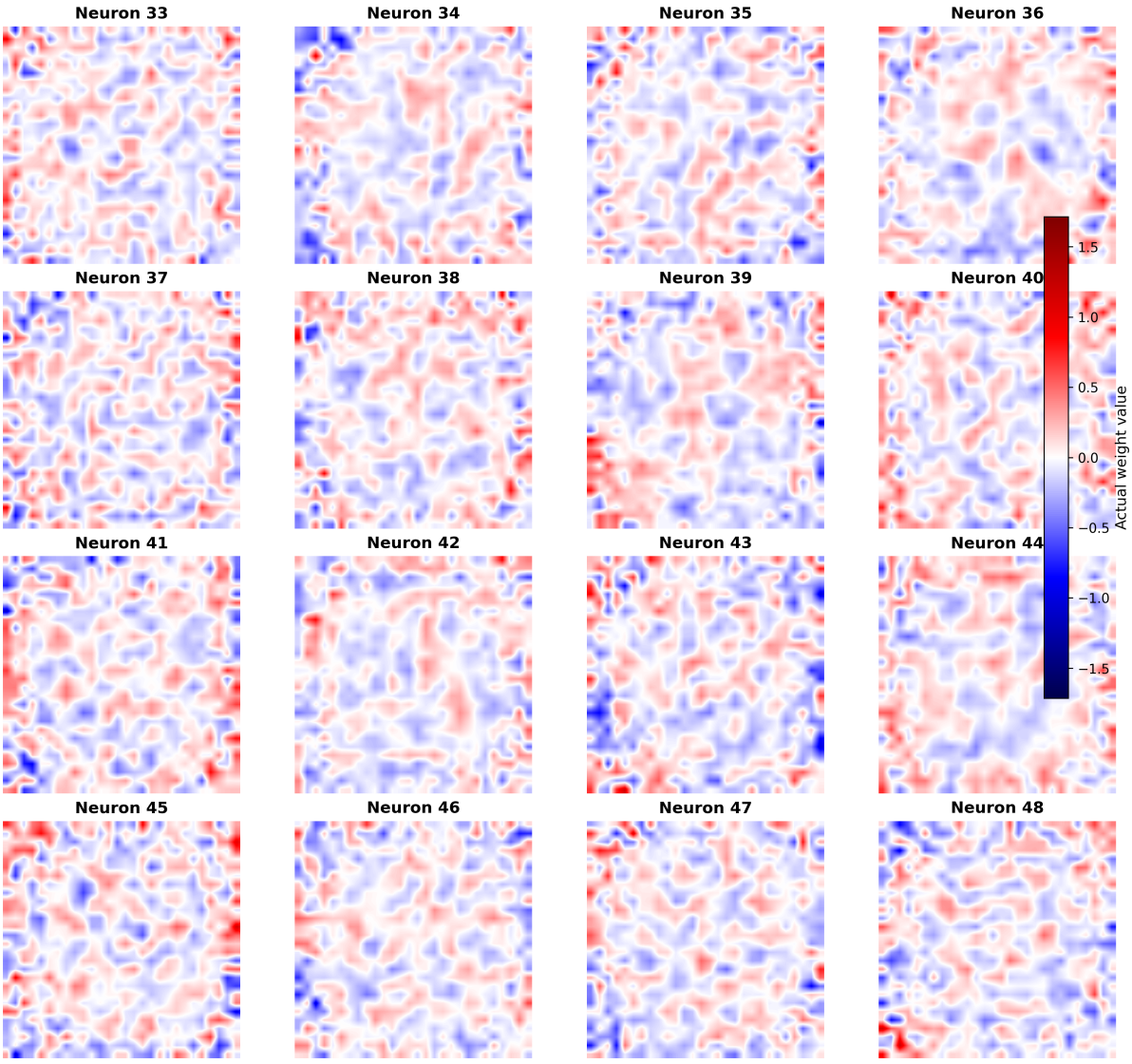

Fig. 47. Input-to-bottleneck weight visualization for neurons 33–48.

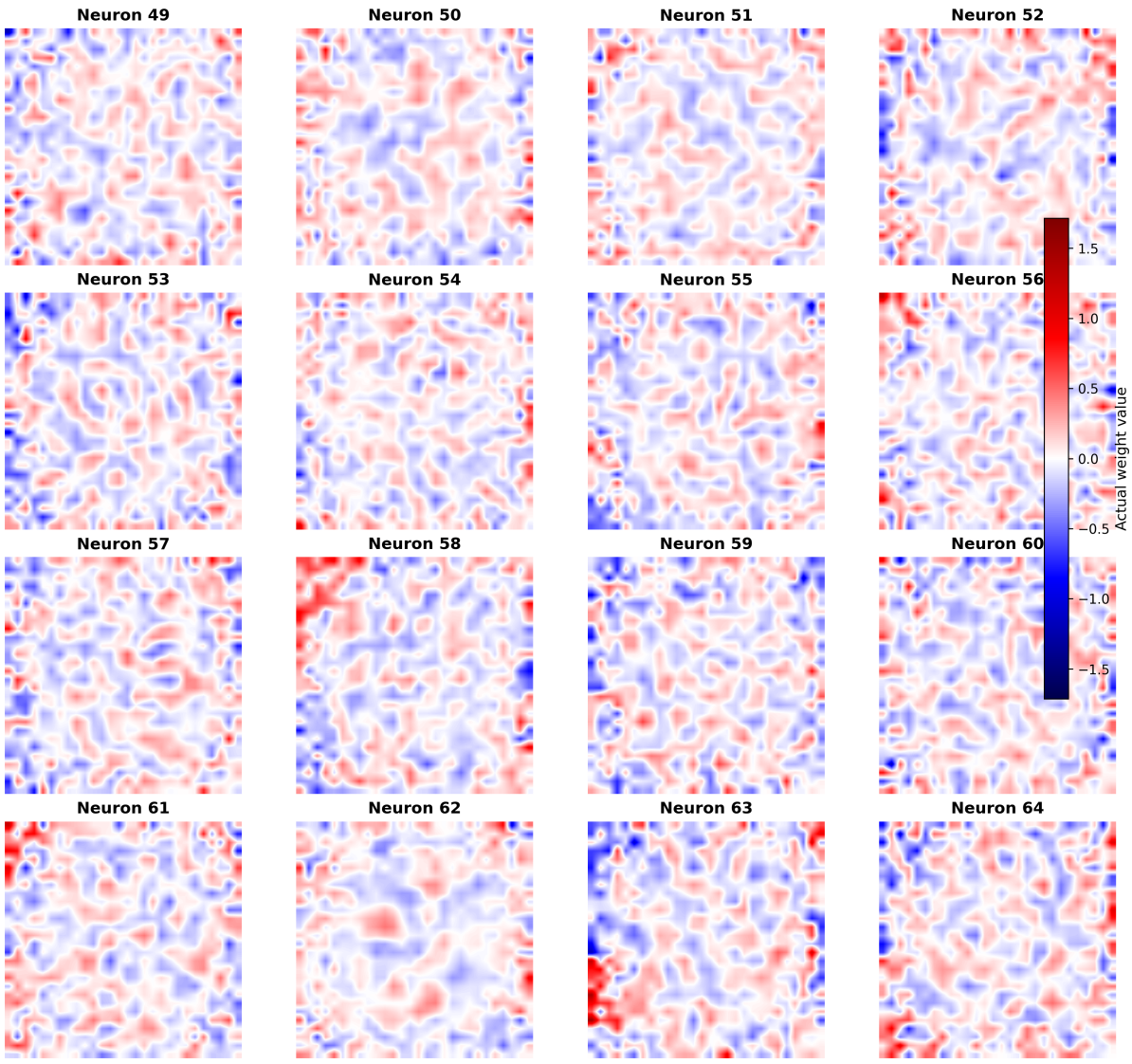

Fig. 48. Input-to-bottleneck weight visualization for neurons 49–64.

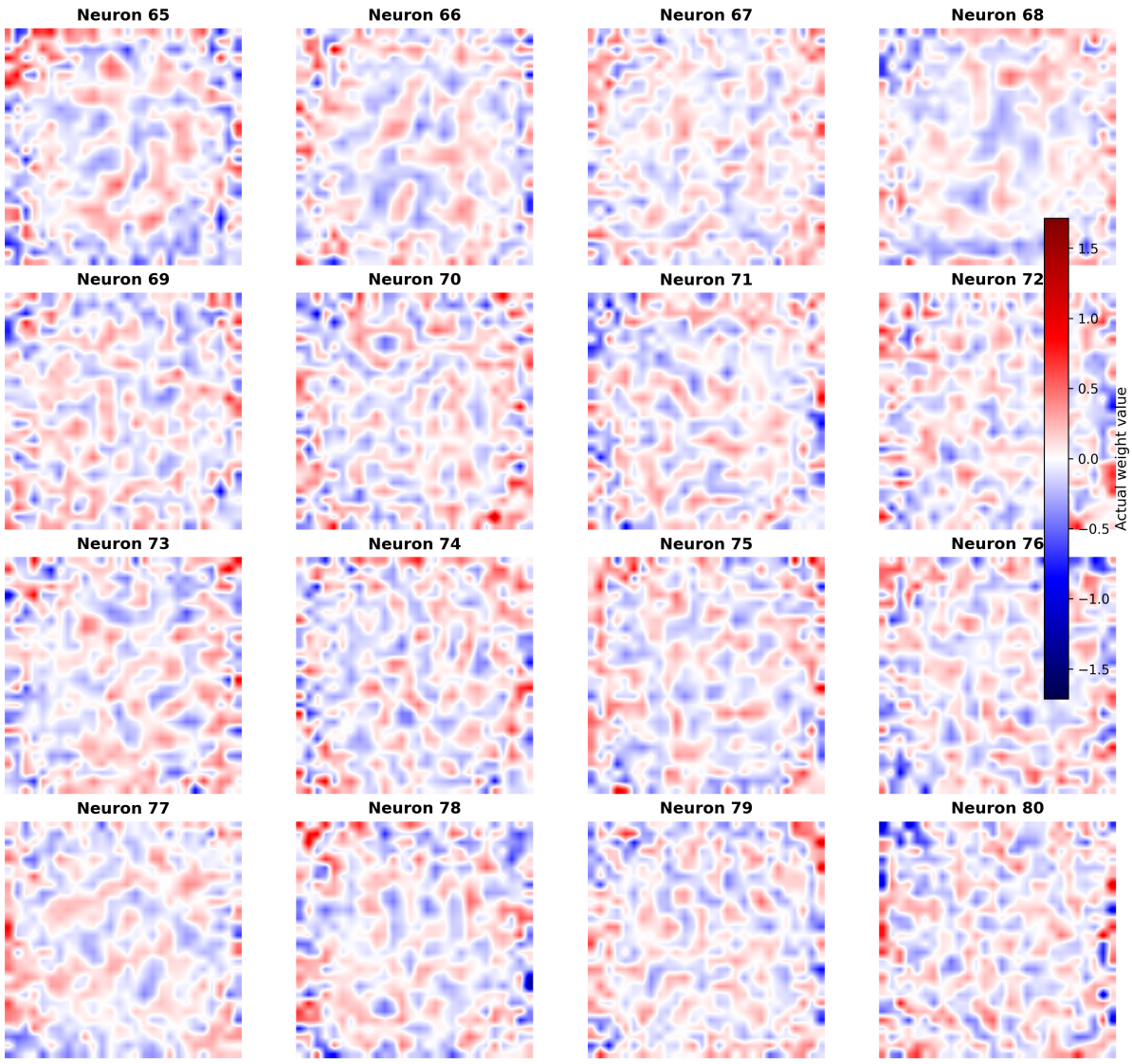

Fig. 49. Input-to-bottleneck weight visualization for neurons 65–80.

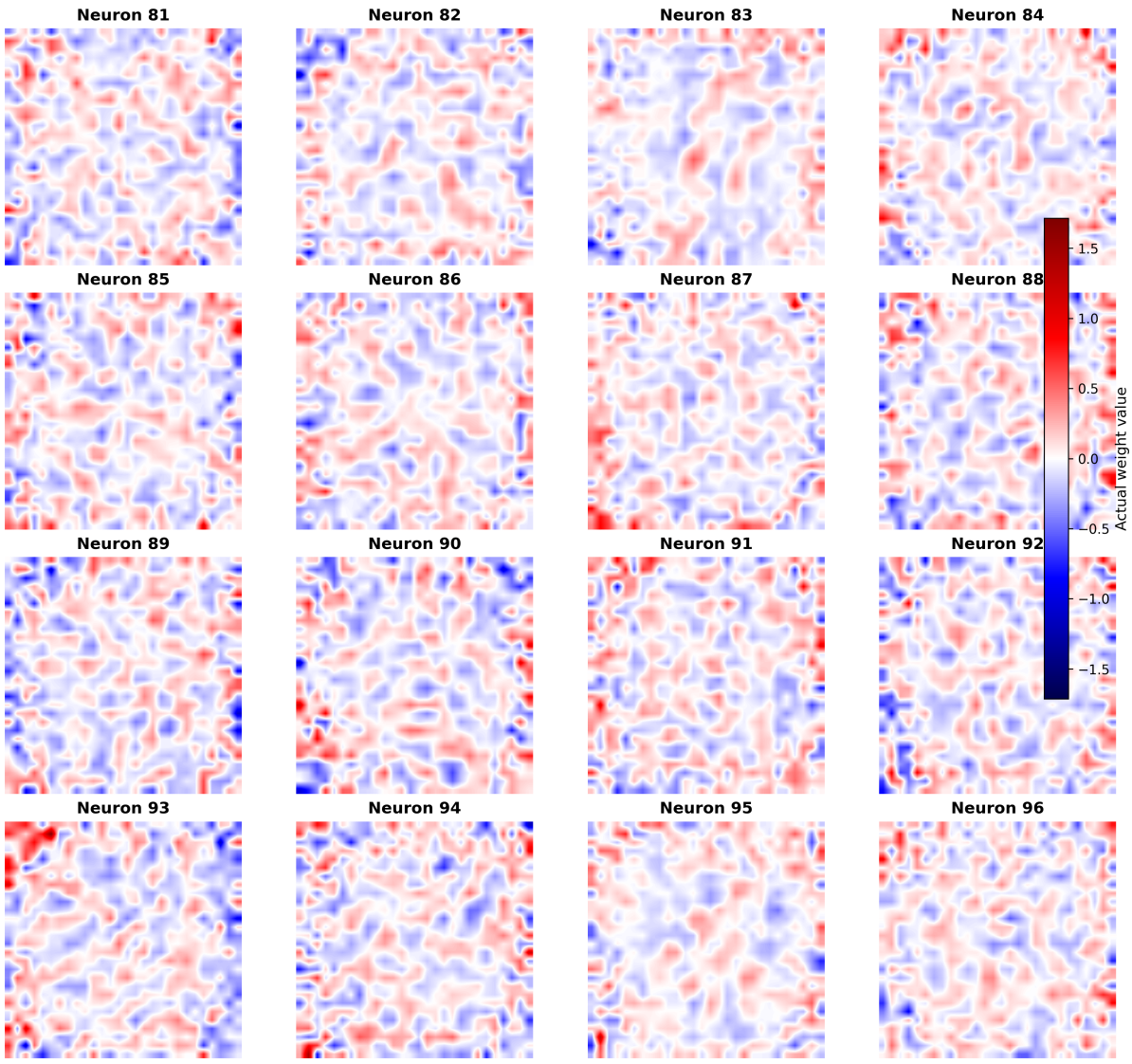

Fig. 50. Input-to-bottleneck weight visualization for neurons 81–96.

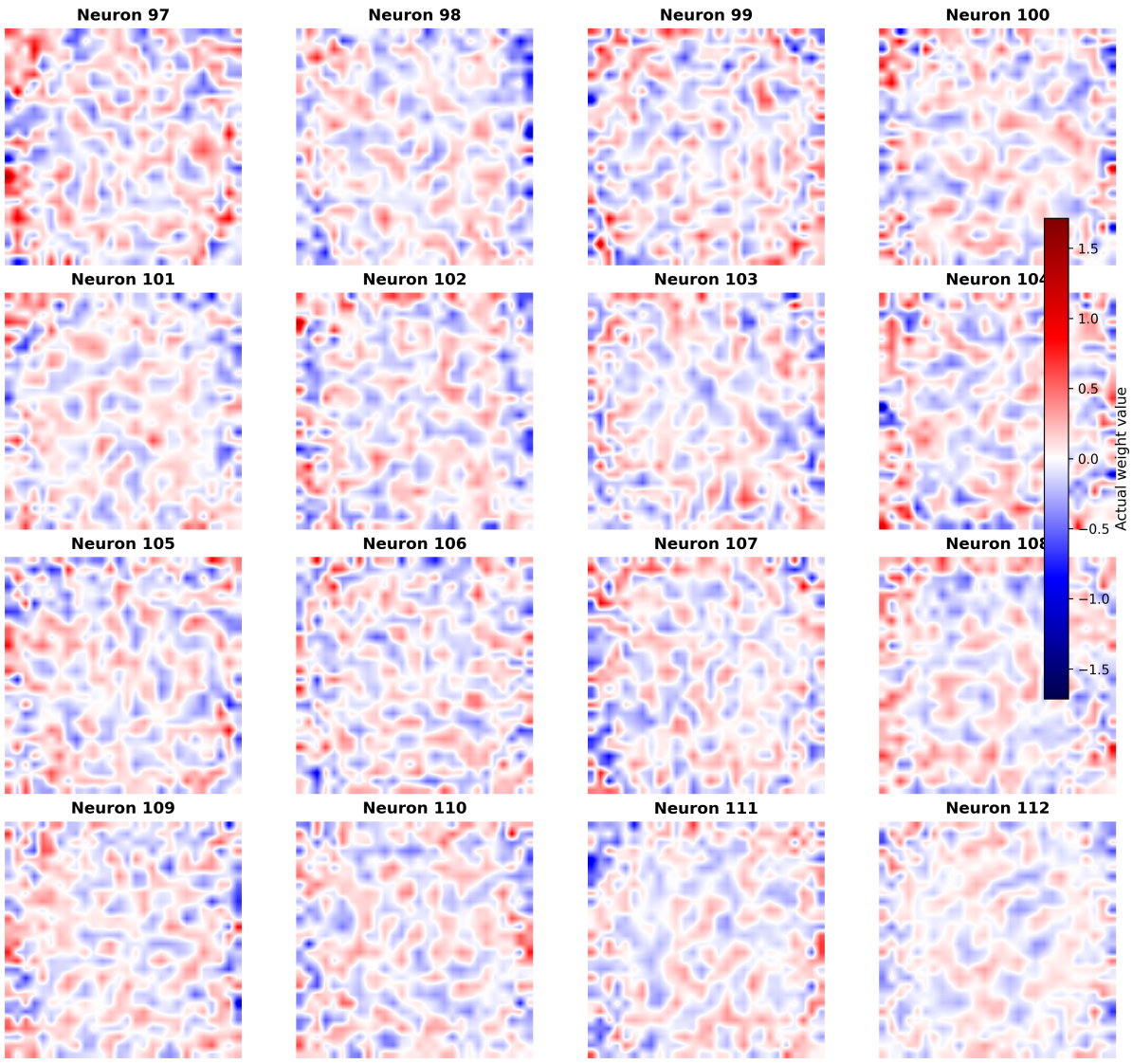

Fig. 51. Input-to-bottleneck weight visualization for neurons 97–112.

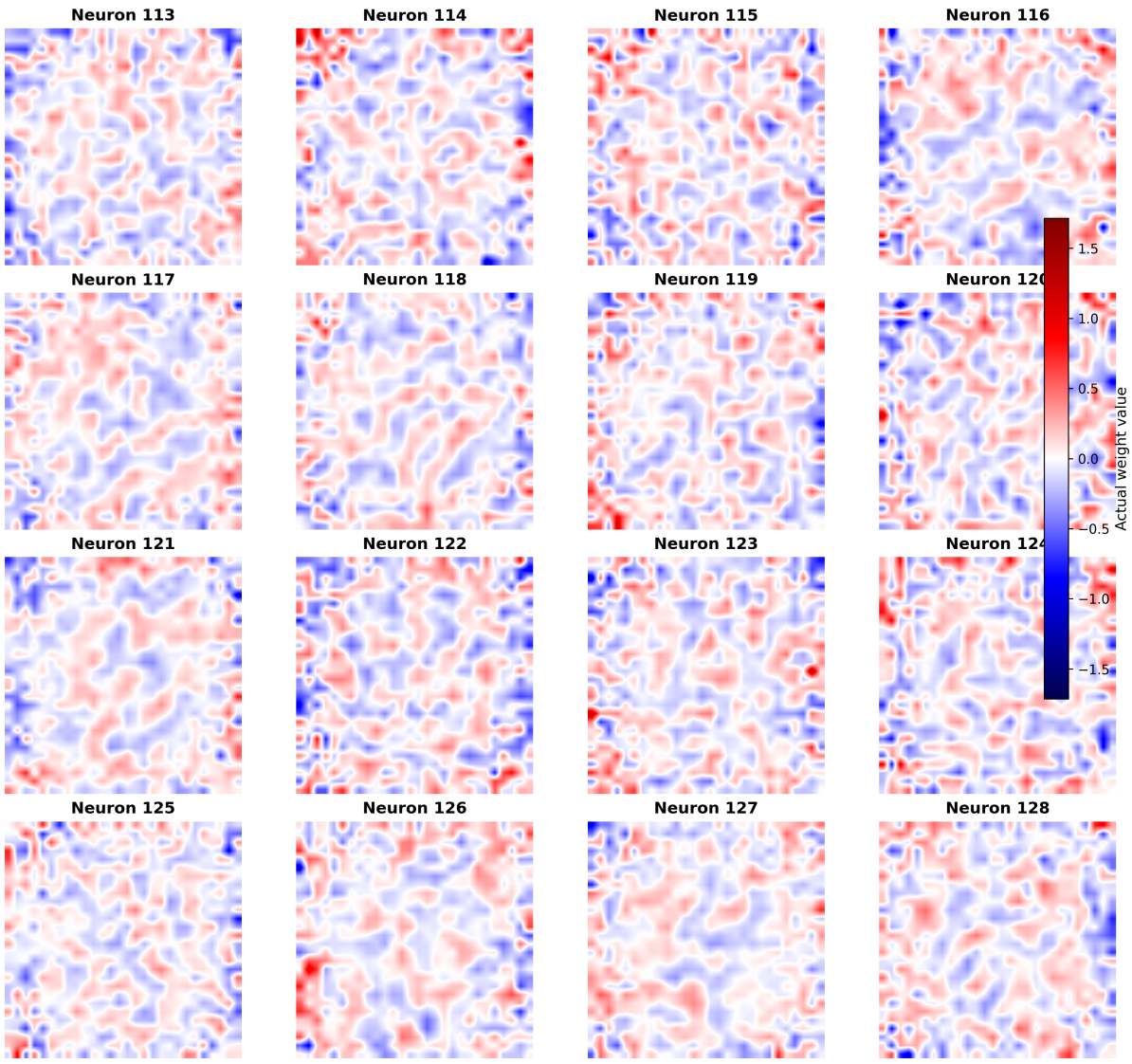

Fig. 52. Input-to-bottleneck weight visualization for neurons 113–128.

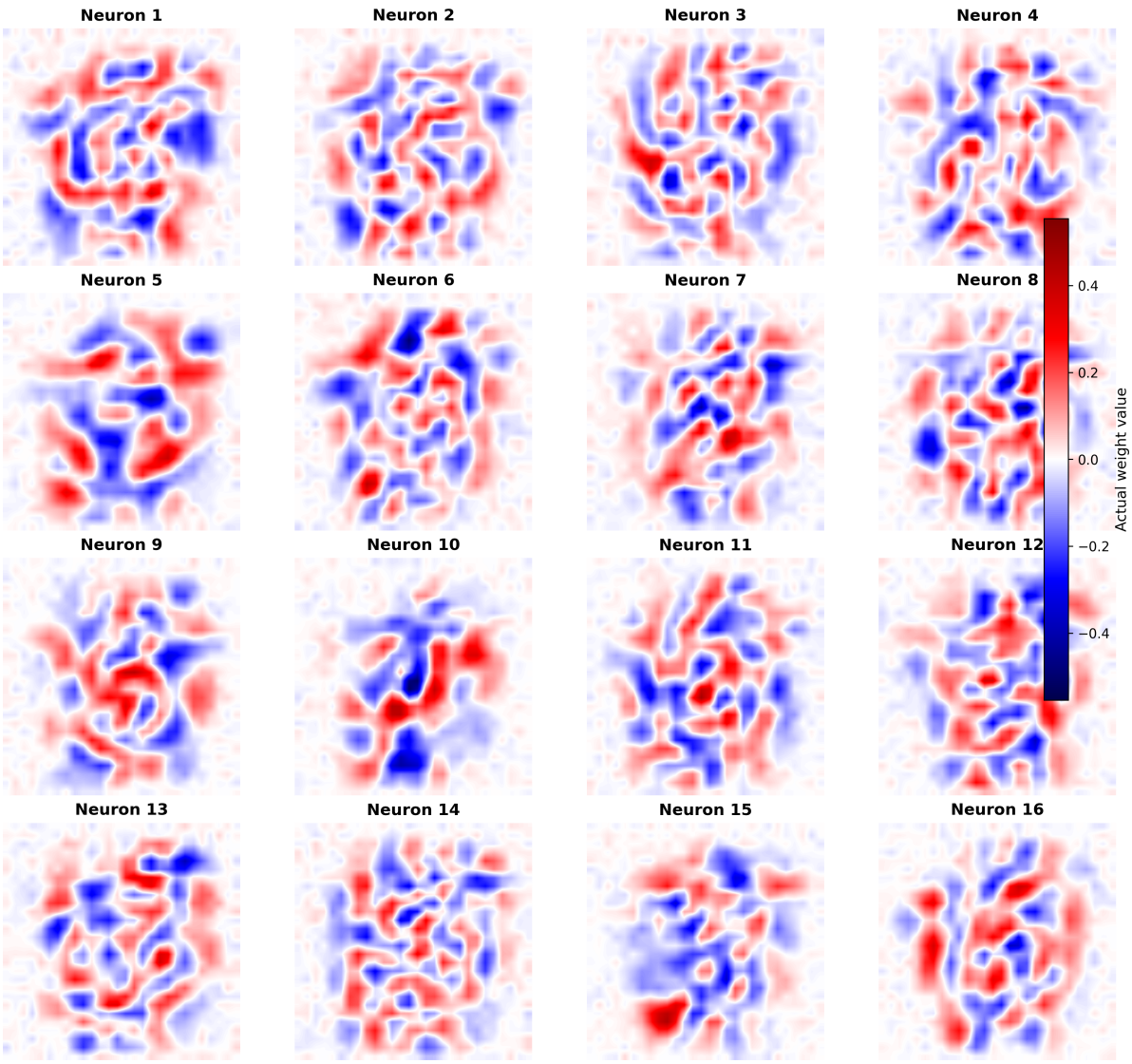

Fig. 53. Input-to-bottleneck weight visualization for neurons 1–16.

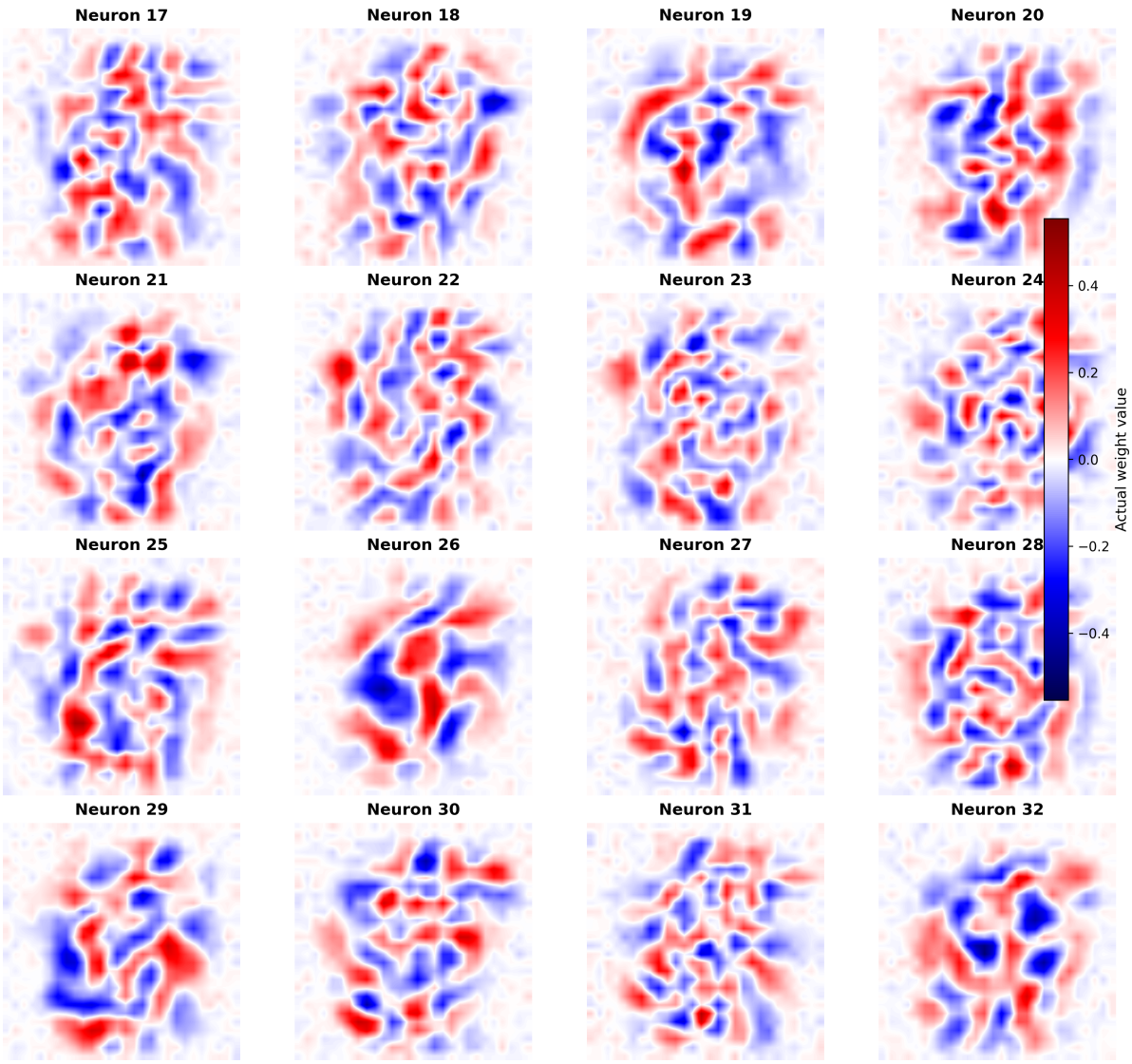

Fig. 54. Input-to-bottleneck weight visualization for neurons 17–32.

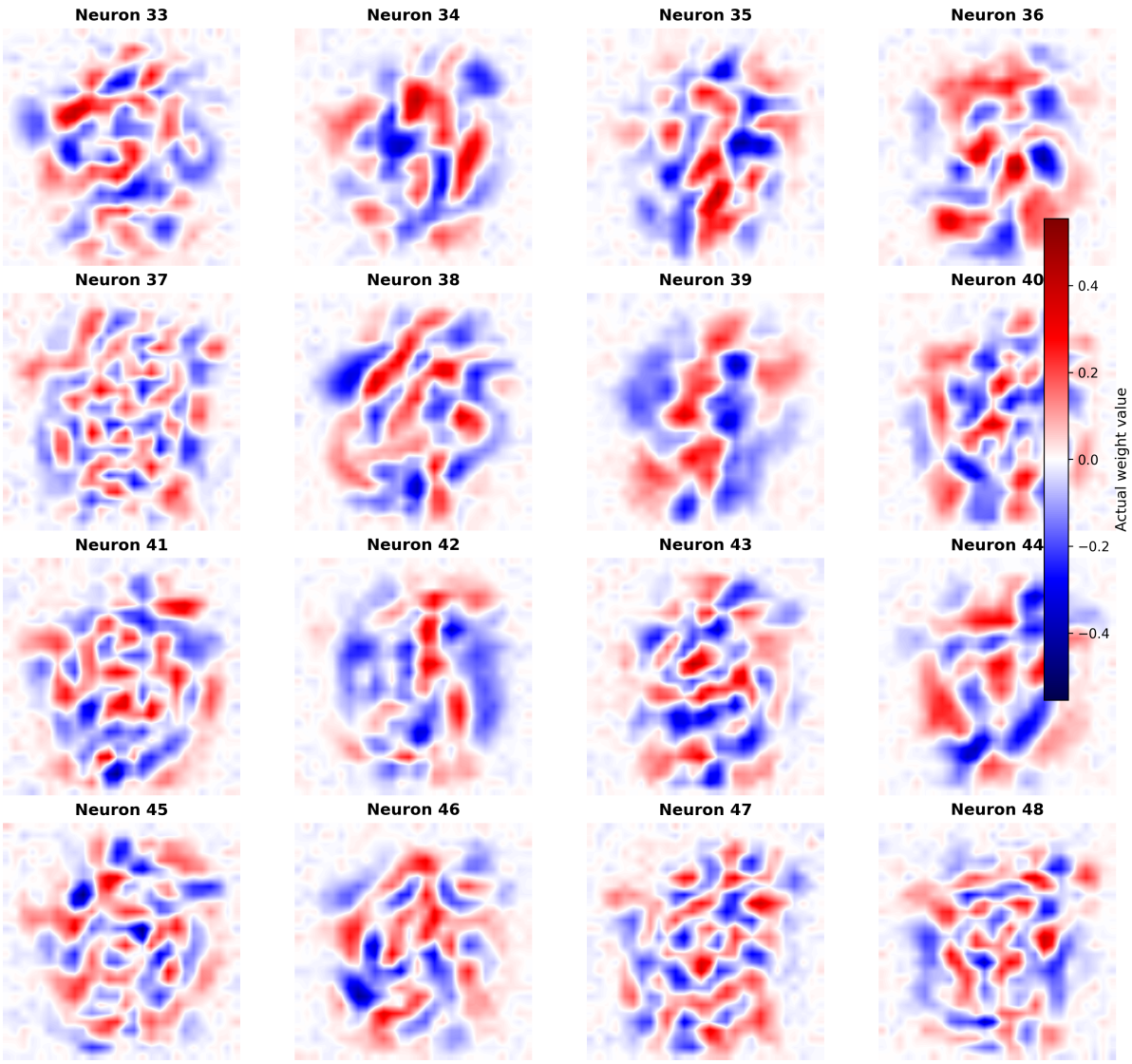

Fig. 55. Input-to-bottleneck weight visualization for neurons 33–48.

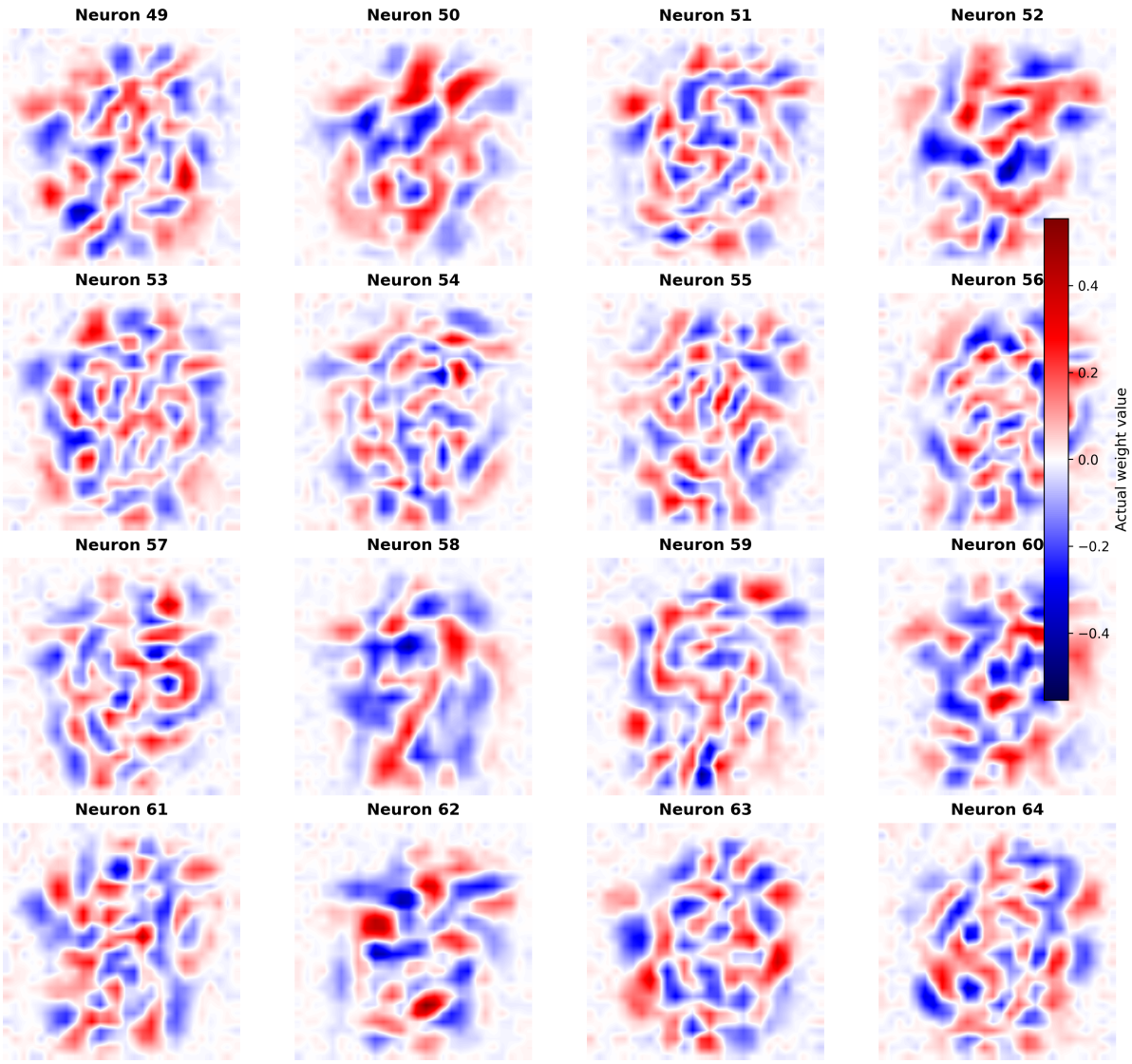

Fig. 56. Input-to-bottleneck weight visualization for neurons 49–64.

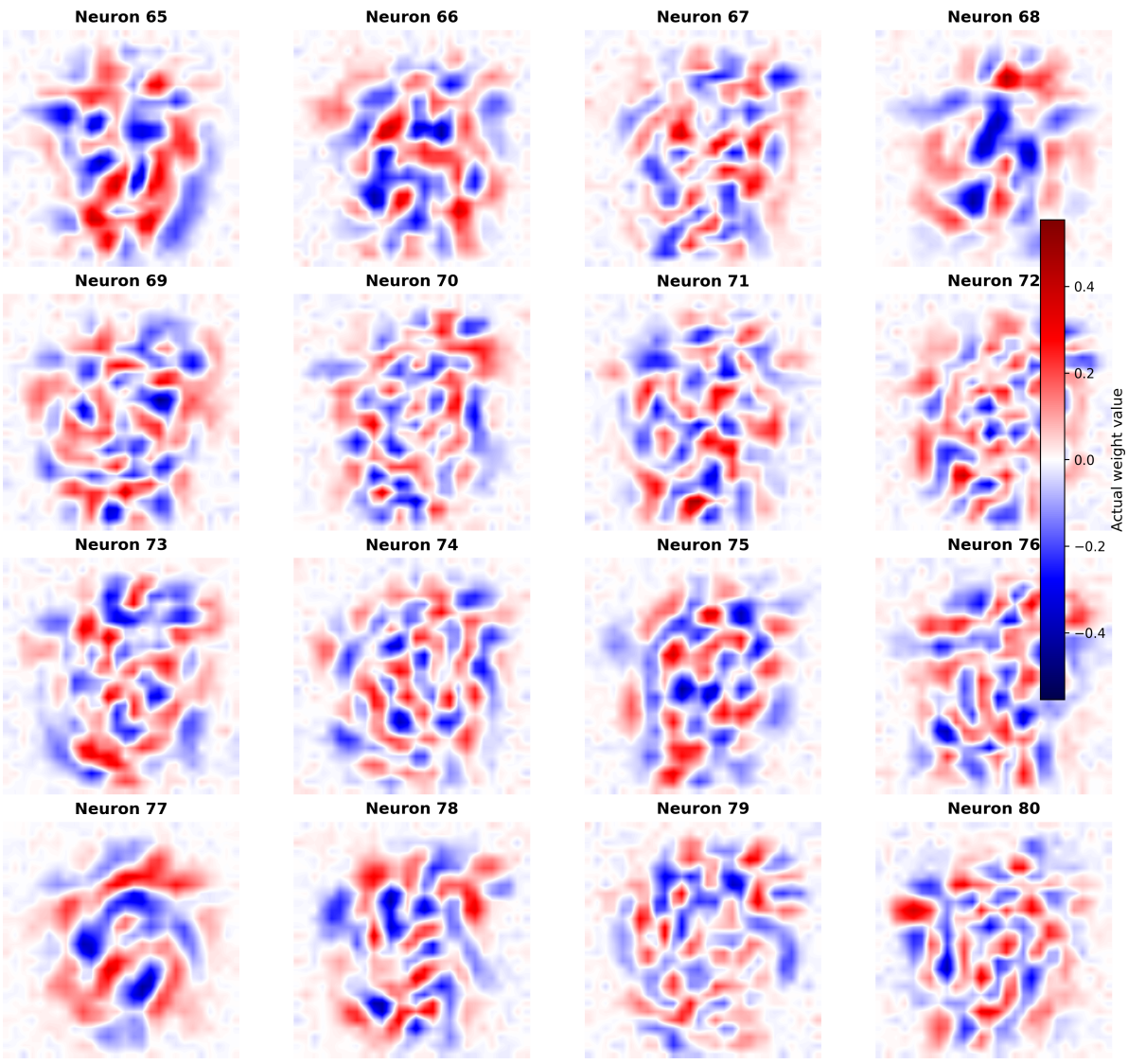

Fig. 57. Input-to-bottleneck weight visualization for neurons 65–80.

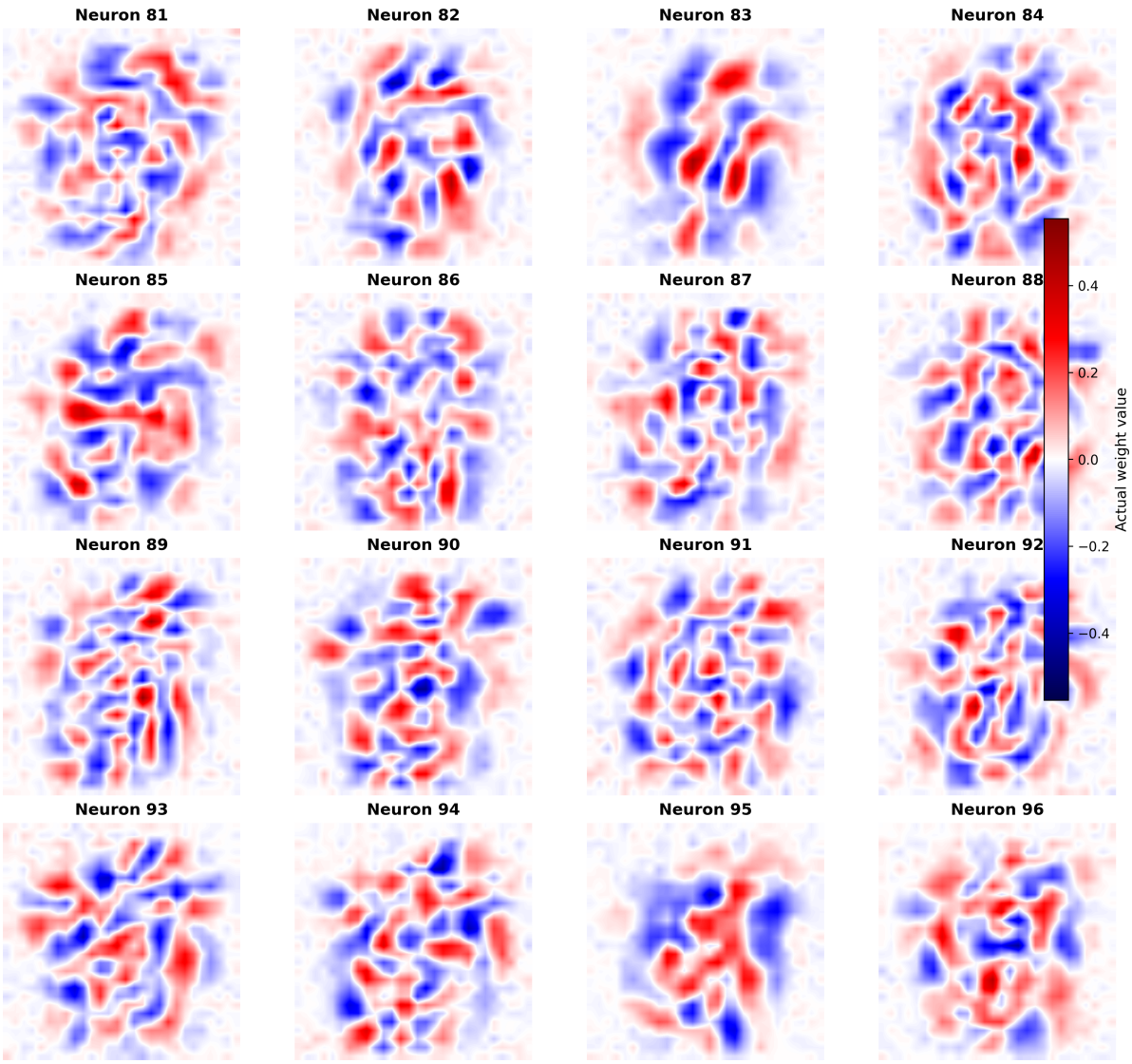

Fig. 58. Input-to-bottleneck weight visualization for neurons 81–96.

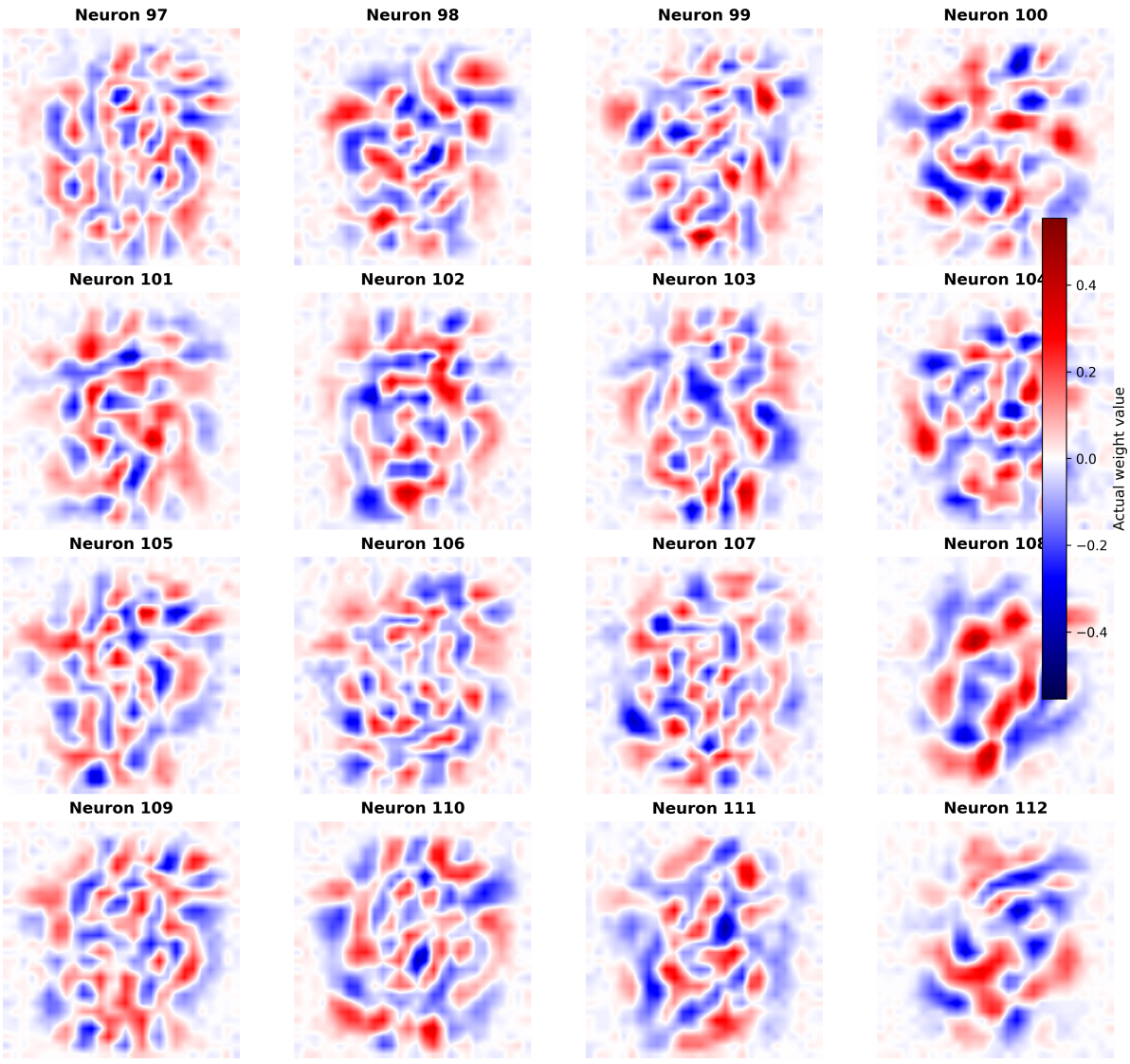

Fig. 59. Input-to-bottleneck weight visualization for neurons 97–112.

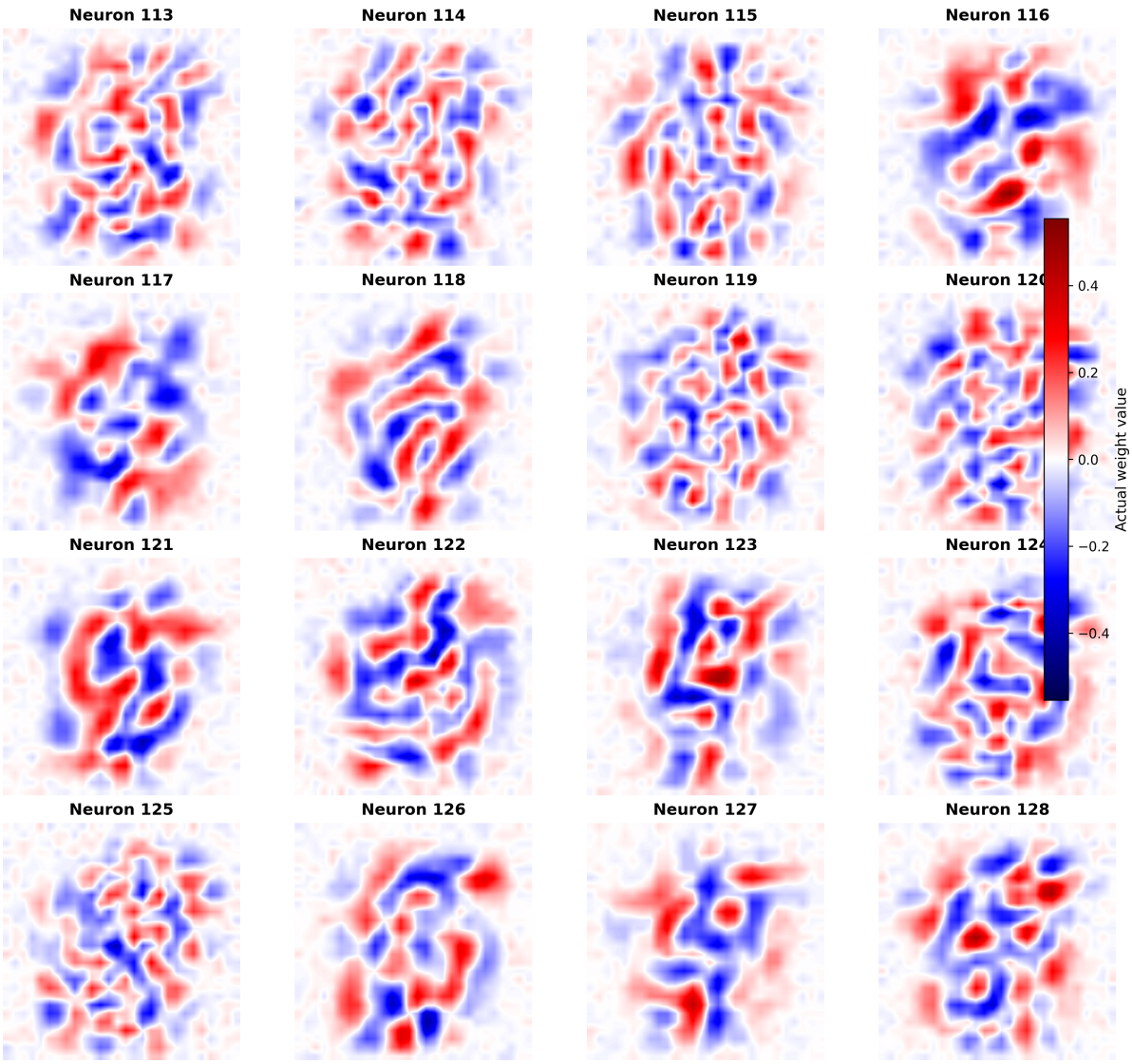

Fig. 60. Input-to-bottleneck weight visualization for neurons 113–128.

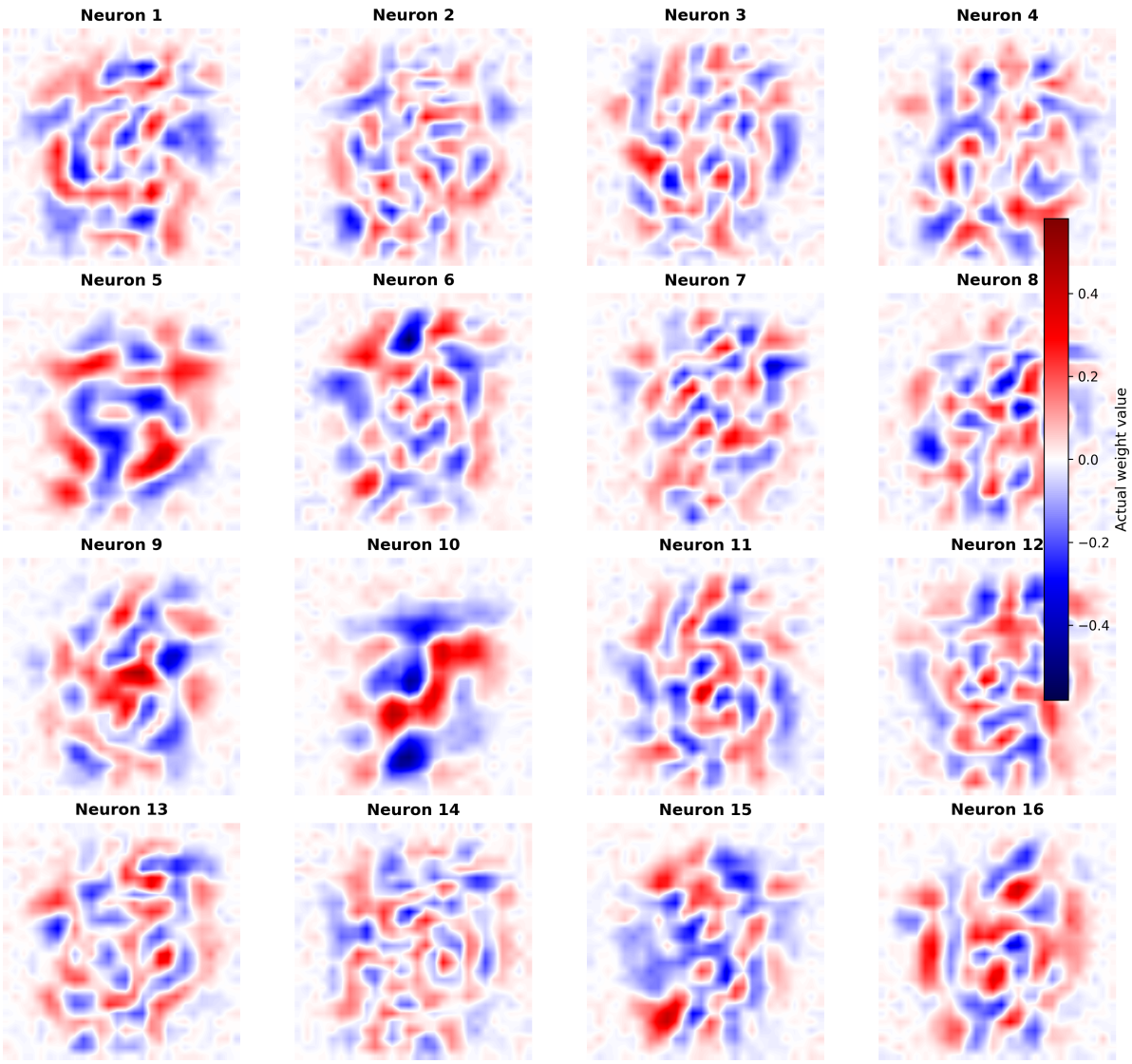

Fig. 61. Input-to-bottleneck weight visualization for neurons 1–16.

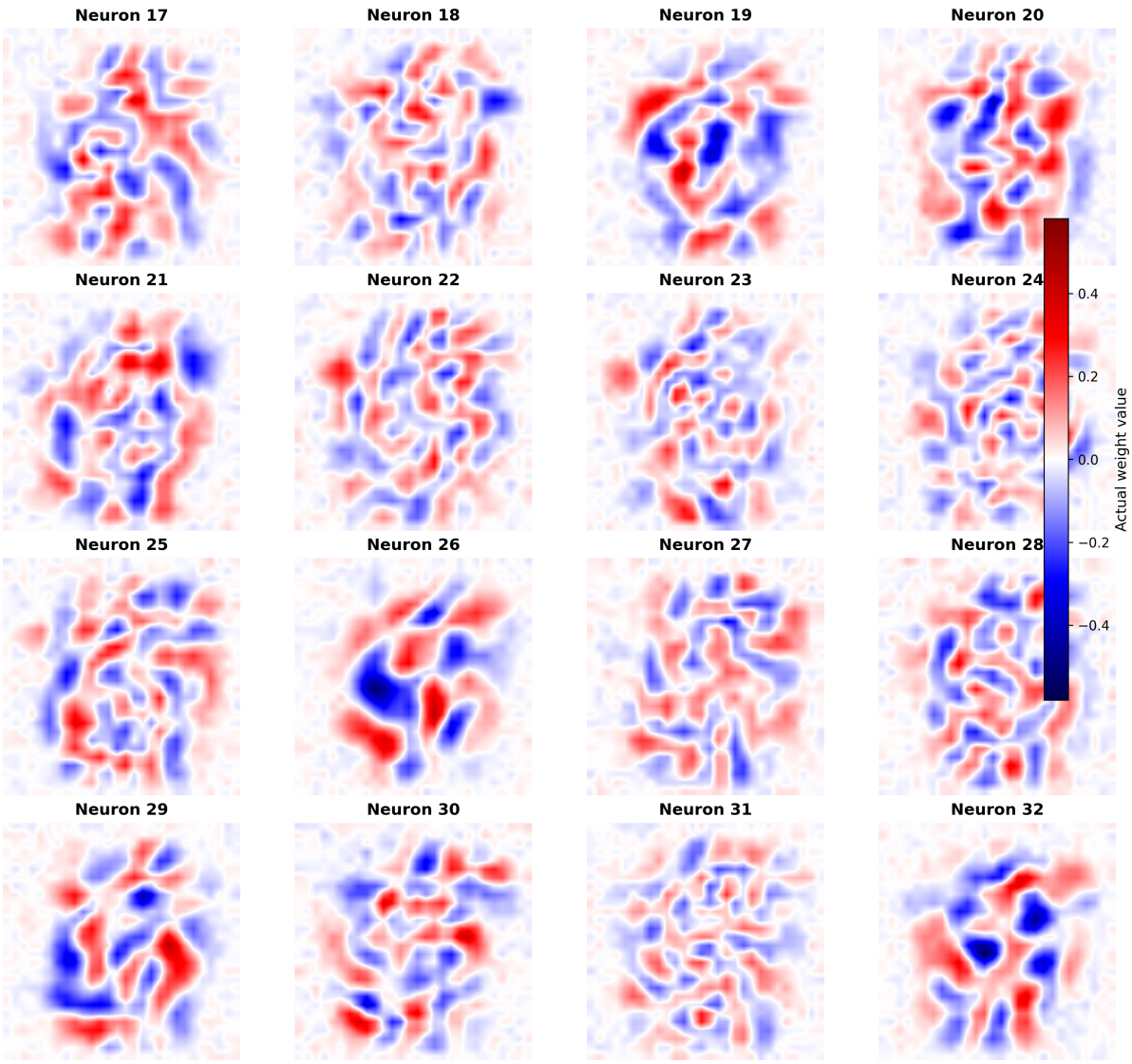

Fig. 62. Input-to-bottleneck weight visualization for neurons 17–32.

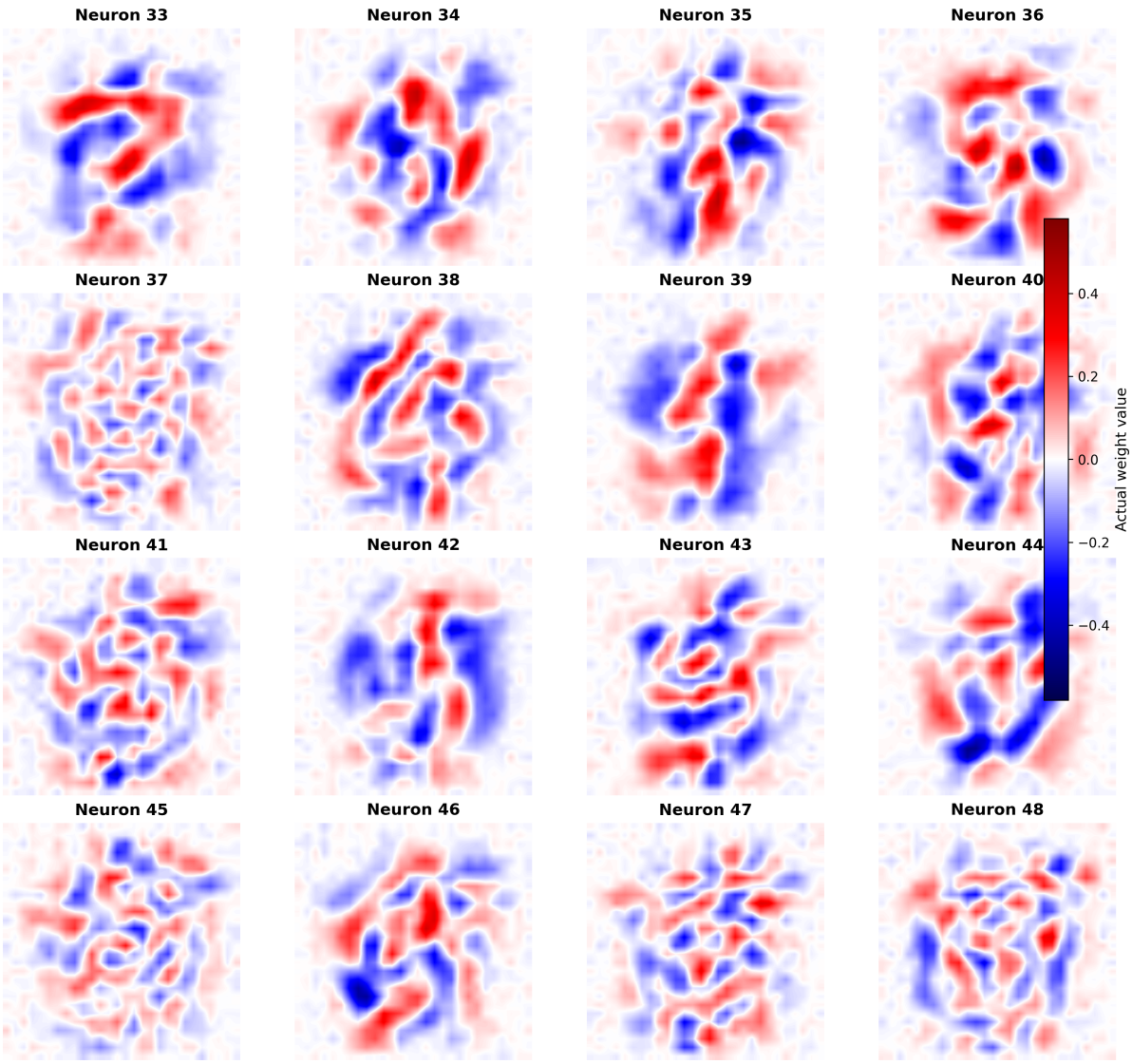

Fig. 63. Input-to-bottleneck weight visualization for neurons 33–48.

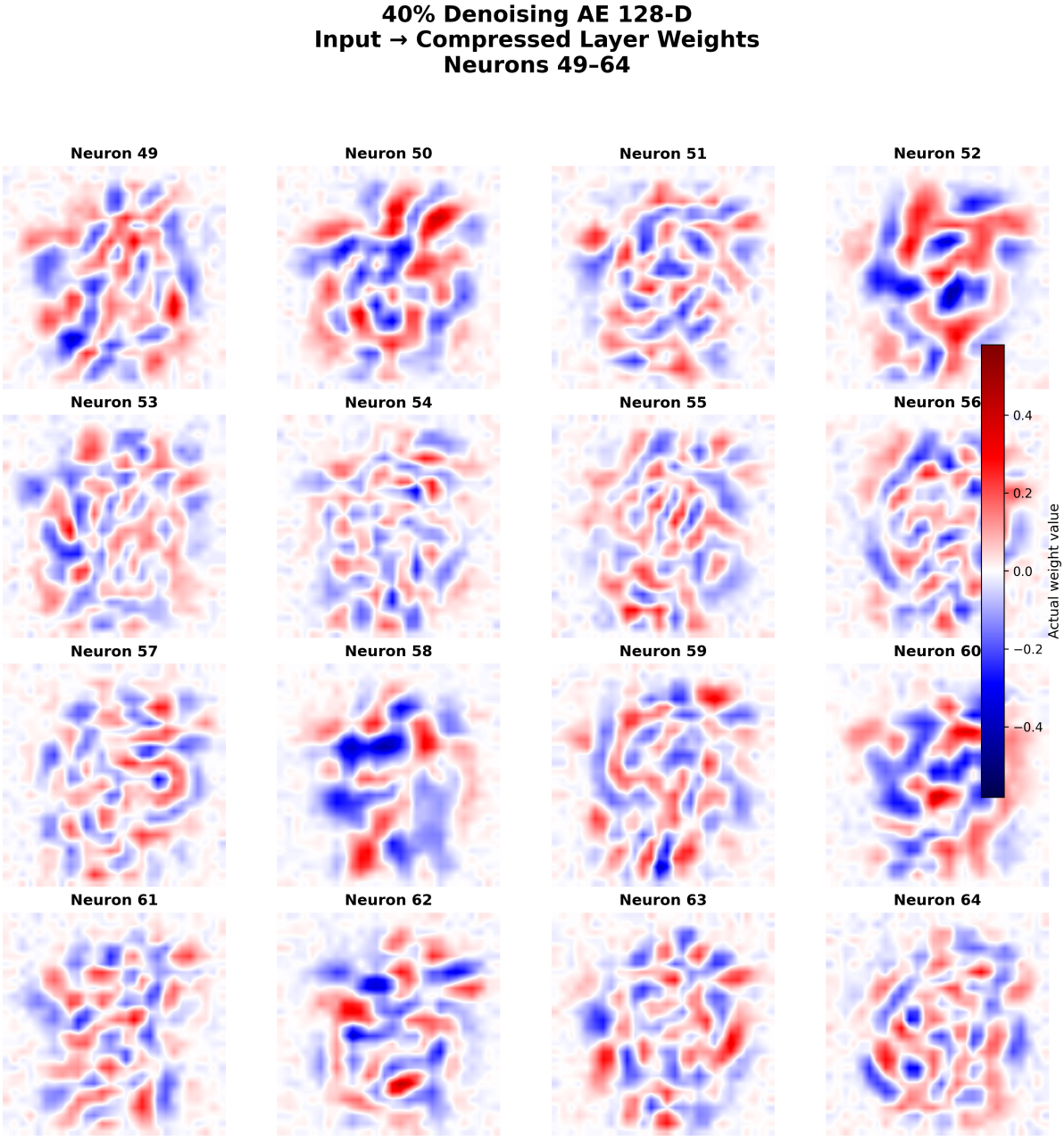

Fig. 64. Input-to-bottleneck weight visualization for neurons 49–64.

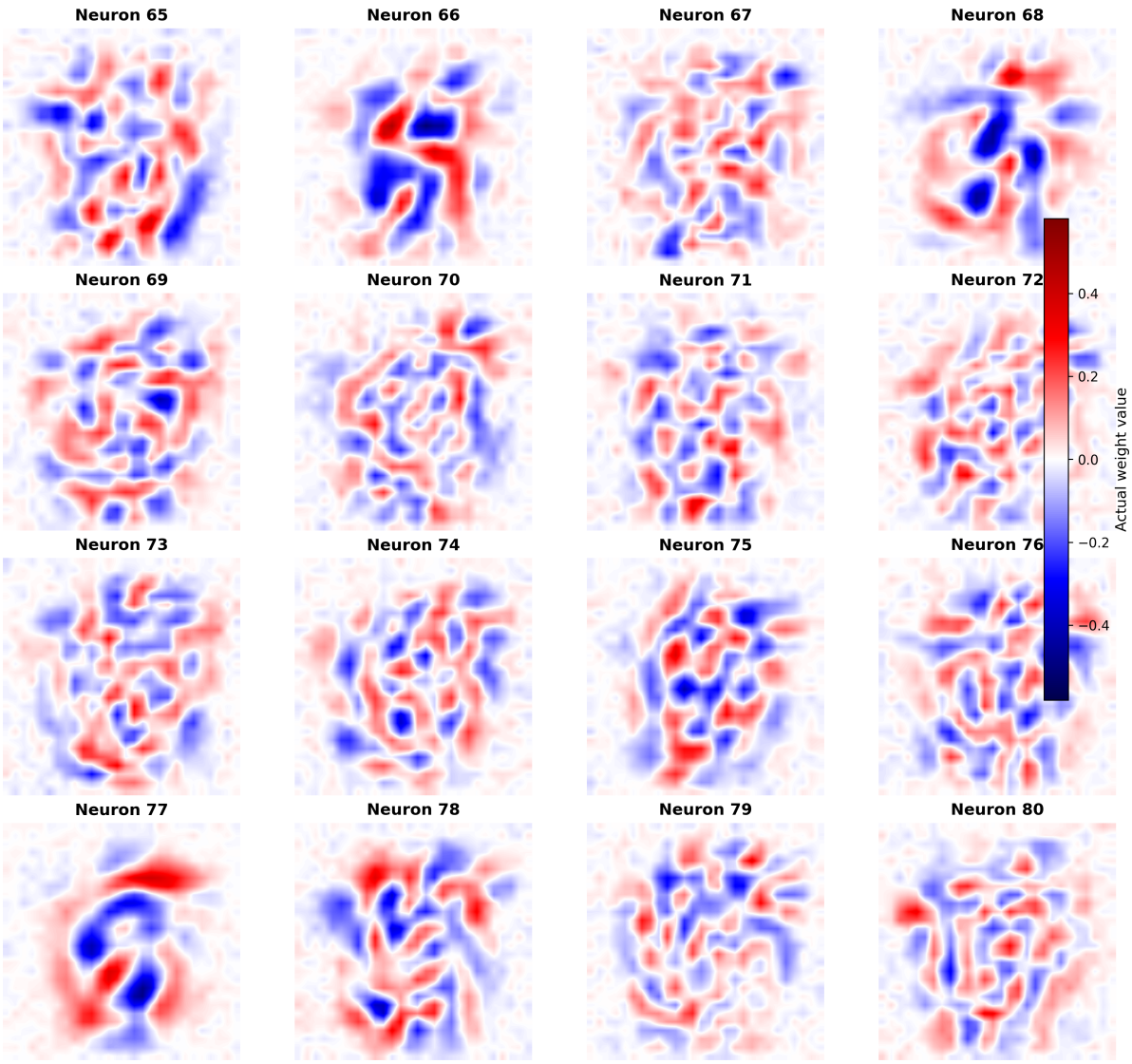

Fig. 65. Input-to-bottleneck weight visualization for neurons 65–80.

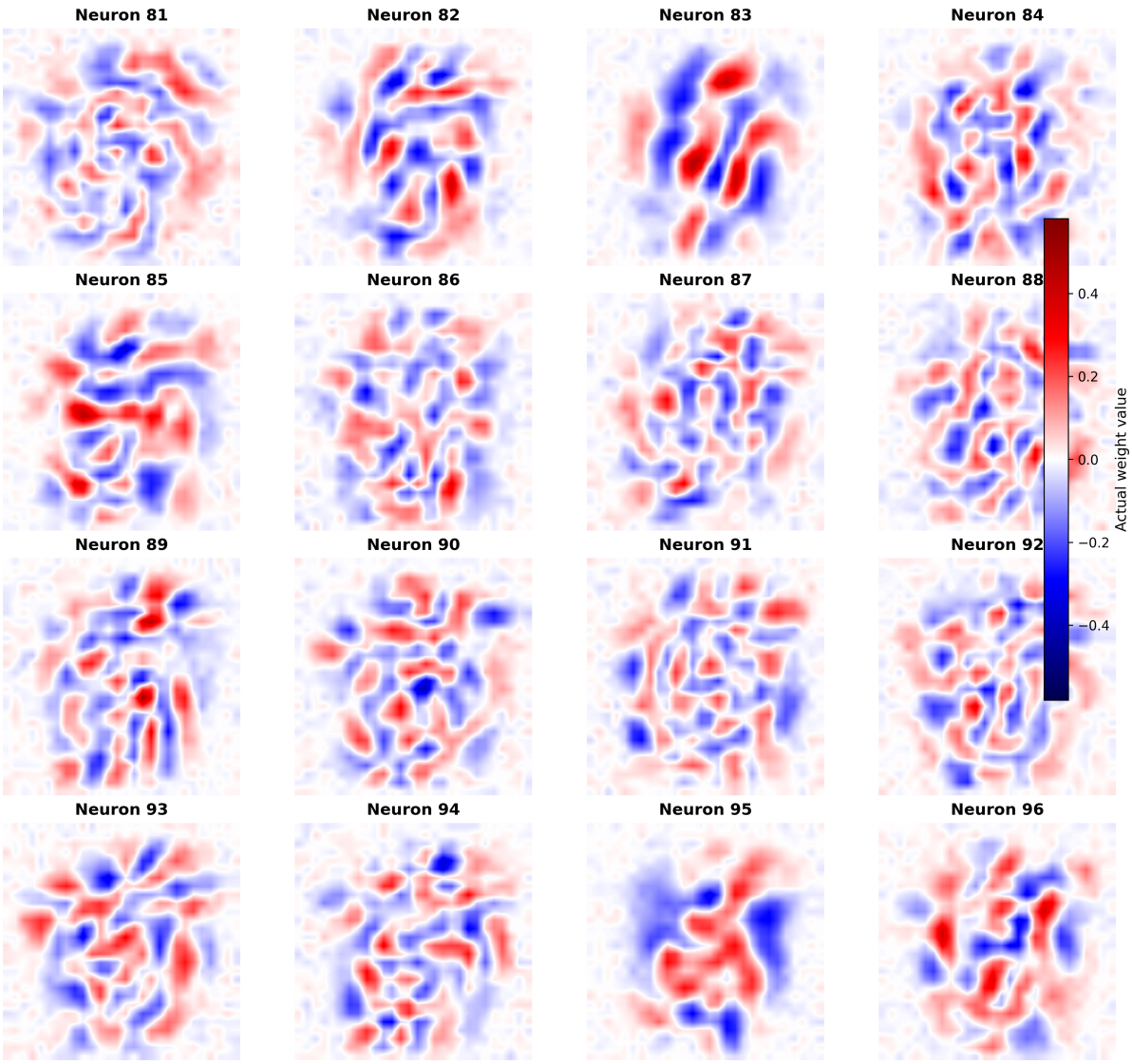

Fig. 66. Input-to-bottleneck weight visualization for neurons 81–96.

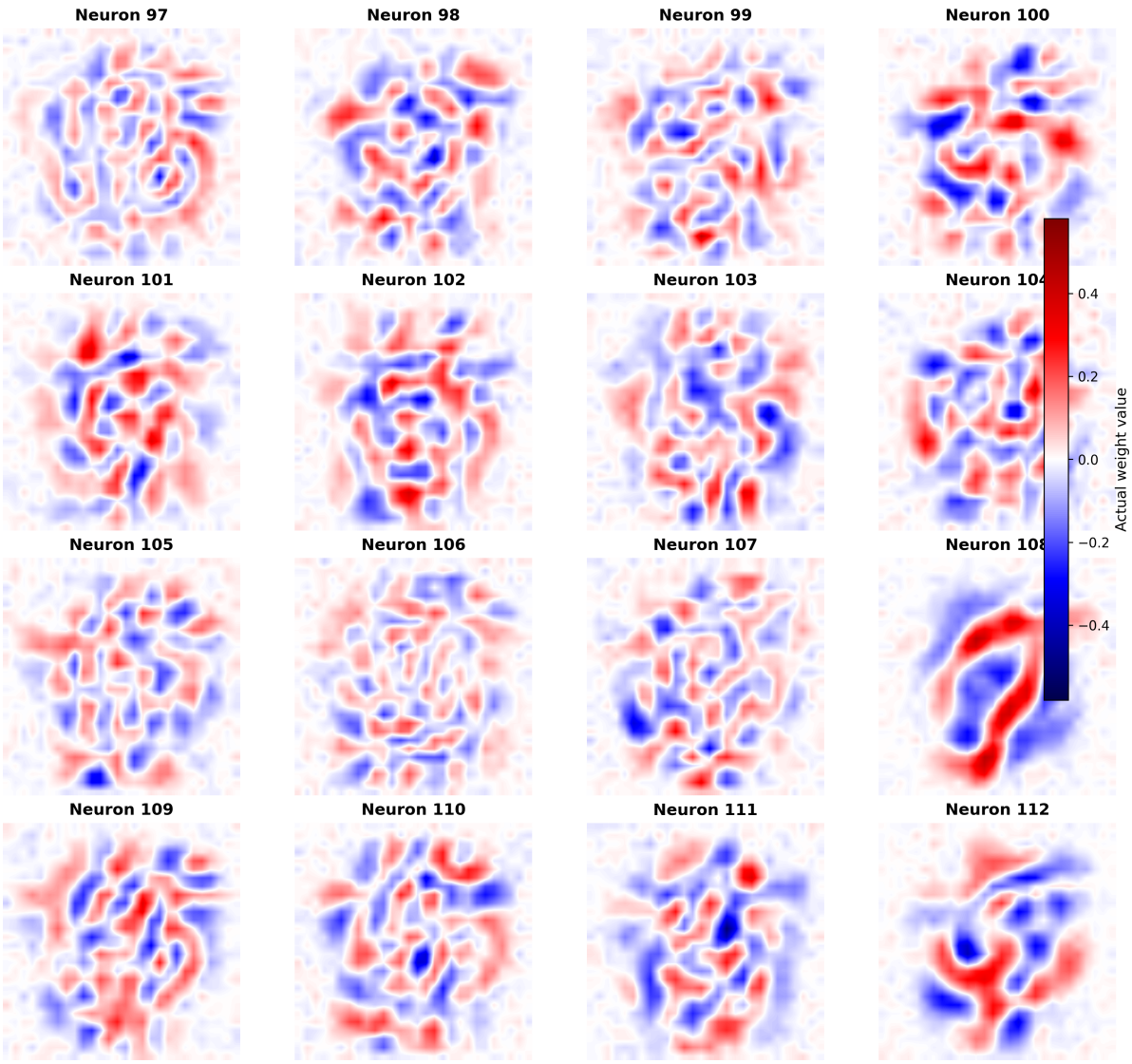

Fig. 67. Input-to-bottleneck weight visualization for neurons 97–112.

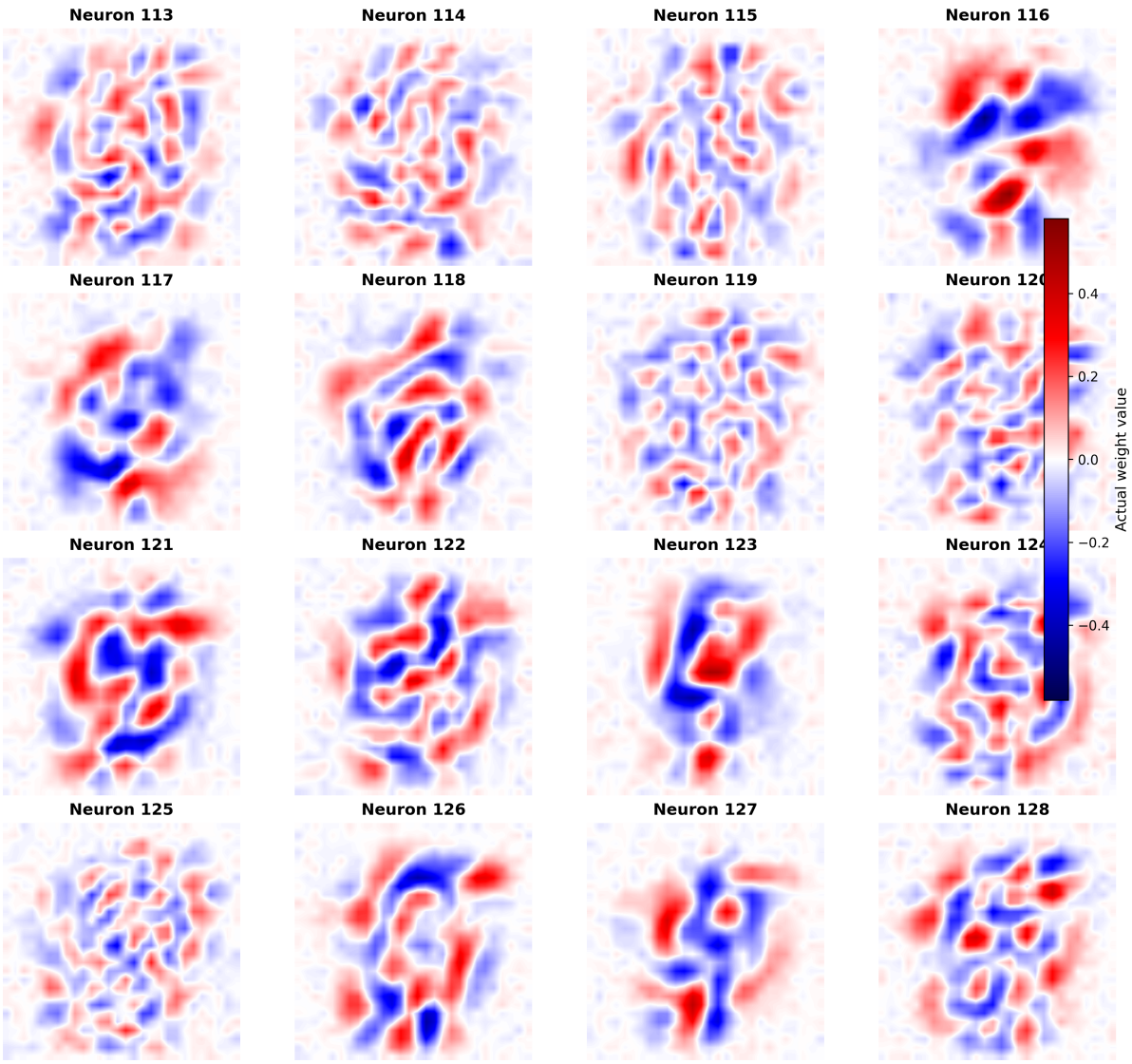

Fig. 68. Input-to-bottleneck weight visualization for neurons 113–128.

IX. Overall Comparison and Discussion

Table X compares the best test results from Tasks 1, 3, 4, and 5 with the Assignment 3 baseline. Among the reported Assignment 4 classification experiments, the 20% DAE gives the highest test accuracy at 98.4453%, followed by the three-hidden-layer AE encoder at 64 dimensions with 98.2872%. PCA reaches 98.2345% at both 64 and 128 dimensions. The one-hidden-layer AE representation reaches 98.1555% at 128 dimensions.

TABLE X
Overall best test classification results.

Method	Best representation	Test acc. (%)
Assignment 3 FCNN baseline	784-D input	98.1291
Task 1: PCA + FCNN	64-D / 128-D	98.2345
Task 3: One-hidden AE + FCNN	128-D	98.1555
Task 4: Three-hidden AE encoder + FCNN	64-D	98.2872
Task 5: 20% DAE + FCNN	128-D	98.4453
Task 5: 40% DAE + FCNN	128-D	98.4190

These results support three cautious observations. First, dimensionality reduction can retain enough information for strong classification while reducing the input size substantially. Second, the latent dimension that minimizes reconstruction error is not necessarily the dimension that maximizes classification accuracy, because reconstruction and classification optimize different objectives. Third, in the denoising comparison, 20% corruption outperforms 40% corruption on both reconstruction MSE and downstream classification accuracy. These conclusions apply to the tested configurations and should not be generalized beyond the experimental setup without further experiments.

X. Conclusion

This assignment compared PCA features, conventional autoencoder, and denoising autoencoder representations for five-class handwritten-digit classification. PCA achieved 98.2345% test accuracy at both 64 and 128 dimensions. The best one-hidden-layer AE representation reached 98.1555%, while the best three-hidden-layer encoder representation reached 98.2872%. The 20% DAE achieved the strongest reported Assignment 4 classification accuracy, 98.4453%, and also reconstructed better than the 40% DAE according to test MSE. The reconstruction examples and weight visualizations provide qualitative complements to the quantitative metrics.

References

[1] CS601T: Deep Learning, "Programming Assignment 3," Department of Computer Science and Engineering, Indian Institute of Technology Dharwad, 2026.

[2] CS601T: Deep Learning, "Autoencoders," Course Lecture Notes, Department of Computer Science and Engineering, Indian Institute of Technology Dharwad, 2026.